

Sequence Variables: A Constraint Programming Computational Domain for Routing and Sequencing

AUGUSTIN DELECLUSE , KU Leuven, Belgium and UCLouvain, Belgium

PIERRE SCHAUS , UCLouvain, Belgium

PASCAL VAN HENTENRYCK , GATECH, USA

Constraint Programming (CP) offers an intuitive, declarative framework for modeling Vehicle Routing Problems (VRP). While classical successor-based CP models can be adapted to handle optional visits or insertion-based heuristics, sequence variables provide a significantly more natural and elegant formulation for these requirements. Building upon our prior work that introduced the initial concept, the main contribution of this article is the complete semantic and operational formalization of sequence variables as a computational domain. Specifically, we formally define the sequence domain and its update operations, and detail the implementation and data structures required to integrate sequence variables into trail-based CP solvers. Furthermore, we introduce consistency levels for associated constraints on this domain alongside specialized global constraints tailored for routing problems. Finally, we demonstrate that sequence variables simplify problem modeling while achieving competitive computational performance on Pickup and Delivery Problems with and without Time Windows, the Dial-a-Ride Problem, and a Prize-Collecting Scheduling Problem.

1 Introduction

The Vehicle Routing Problem (VRP) has many variants (Braekers et al. 2016). Constraint Programming (CP) is a widely used approach for solving them (Kilby and Shaw 2006), as its declarative modeling paradigm allows for flexible adaptation to constraints and objectives. However, existing CP approaches struggle to handle optional visits and insertion-based search strategies, which are crucial to quickly obtain high-quality solutions. As a motivating example for using sequence variables, we consider the Dial-A-Ride Problem (DARP).

Intuitively, the DARP involves routing a fleet of vehicles to satisfy a set of transportation requests for people. Each request corresponds to a user's need to be picked up at an origin location and dropped off at a destination. When boarding, the passenger(s) occupy a certain amount of capacity in the vehicle (which may vary, for instance, if a wheelchair is required or if there are multiple passengers to load), and this capacity is freed upon reaching their destination. Formally, the input to DARP consists of the number m of vehicles, n requests, the maximum ride time L for customers, the maximum route duration D , and the planning time horizon T , i.e., the hours between which the vehicles can operate. A DARP is defined on a complete graph $G = (V, E)$ where $V = \{v_0, v_1, \dots, v_{2n}\}$ is the set of nodes and $E = \{(v_i, v_j) : v_i, v_j \in V, i \neq j\}$ is the set of edges. Node v_0 denotes the depot. Each request i ($1 \leq i \leq n$) consists of a pair of nodes (v_i, v_{i+n}) . For each node $v_j \in V$, we associate a service duration $s_j \geq 0$, a load change q_j and a time window $[e_j, l_j]$. We also have $s_0 = 0$, $q_0 = 0$, and $e_0 = 0$, $l_0 = T$. The distance between nodes v_i and v_j is denoted by $d_{i,j}$. For each request i , we denote its pickup node by $r_i^+ = v_i$ (where $q_i > 0$) and its delivery node by $r_i^- = v_{i+n}$ (where $q_{i+n} = -q_i$). Given these definitions, a DARP consists in finding a route for each of the m vehicles such that:

- (1) The route begins and ends at the depot;
- (2) The load of a vehicle never exceeds its capacity c ;
- (3) The total duration of each route (i.e., the difference between the end time and the start time at the depot) never exceeds D ;

*Corresponding Author.

Authors' Contact Information: Augustin Delecluse , ORCID: 0000-0001-6285-6515, augustin.delecluse@kuleuven.be, KU Leuven, Leuven, Belgium and UCLouvain, Louvain-La-Neuve, Belgium; Pierre Schaus , ORCID: 0000-0002-3153-8941, pierre.schaus@uclouvain.be, UCLouvain, Louvain-la-Neuve, Belgium; Pascal Van Hentenryck , ORCID: 0000-0001-7085-9994, pascal.vanhentenryck@isye.gatech.edu, GATECH, Atlanta, USA.

- (4) For each request i , r_i^+ and r_i^- are serviced by the same vehicle and r_i^- is visited after r_i^+ ;
- (5) The ride time of any customer i (i.e., the difference between the serving time at the delivery node r_i^- and the departure time at the pickup node r_i^+) does not exceed L ;
- (6) For each node v_j , the starting time of its service lies between $[e_j, l_j]$; and
- (7) The total routing cost of all vehicles is minimized.

Two CP models and their search procedures are given next, illustrating the $CP = model + search$ paradigm.

Comparison of CP formulations. To provide a clear high-level understanding, a classical successor-based CP model (Equations 1–12) and a model using the proposed sequence variables (Equations 13–22) are presented in parallel below to highlight their fundamental differences. Both models use standard mathematical notation, and the precise semantics of the sequence-based global constraints are detailed later in Section 7. For both models, let $K = \{1, \dots, m\}$ and $R = \{r_1, \dots, r_n\}$ denote the sets of vehicles and requests, respectively. The set of nodes V and the distance matrix d are extended to introduce nodes representing the start (α_k) and end (ω_k) nodes for each vehicle $k \in K$, which are duplicates of the depot. We write $V_{\text{end}} = \{\omega_k \mid k \in K\}$ and $V_{\text{tr}} = V \setminus \{\alpha_k, \omega_k \mid k \in K\}$ for the end nodes and transportation nodes, respectively.

The successor model relies on a successor variable $\mathbf{Succ}_v$ for each node v , representing the node visited immediately after v . First, the objective (1) minimizes the total distance between nodes and their successors. To ensure that these predecessor-successor relationships form a valid set of disjoint routes without disconnected sub-tours, the model connects all routes into a single giant Hamiltonian circuit by chaining the end node of vehicle k to the start node of vehicle $k + 1$ (2). This allows the use of a single global *circuit* constraint (3) over all nodes. Time tracking is achieved by local channeling constraints that propagate time to the successor $\mathbf{Succ}_v$ (4), and time windows are explicitly enforced via (5). Similarly, load is initialized to zero (6) and tracked along the route (7), with the vehicle capacity explicitly enforced via (8). Vehicle assignment is handled by channeling the vehicle along the route (9) and pairing pickup and delivery nodes (10).

In contrast, the sequence variable model directly represents the route of each vehicle as an explicitly ordered sequence of nodes. It relies on an insertion-based sequence variable $\mathbf{Route}_k$ for each vehicle $k \in K$. First, the objective (13) minimizes the sum of distances $\mathbf{Dist}_k$ linked to each route via the *Distance* constraint (14). Structurally, every transportation node $v \in V_{\text{tr}}$ must be visited exactly once (15), and the sequence is explicitly constrained to start and end with the designated depot nodes via (16) and (17). Because a sequence variable inherently maintains the total order of visits, it naturally guarantees valid routes without the need for a *circuit* constraint. This explicit ordered structure allows the use of specialized global constraints that operate directly on the entire sequence. For instance, time tracking is managed by the *TransitionTimes* constraint (18), with time windows explicitly enforced via (19). Similarly, load tracking and vehicle assignment are handled by the *Cumulative* constraint (20), which maintains the capacity limits, ensures that the pickup and delivery nodes of a request are serviced by the same vehicle, and enforces that the delivery occurs after the pickup.

Finally, the maximum ride time and route duration constraints (11)–(12), (21)–(22) are identical in both models.

Successor Model

$$\min \sum_{v \in V \setminus V_{\text{end}}} d_{v, \text{Succ}_v} \quad (1)$$

$$\text{Succ}_{\omega_k} = \alpha_k \bmod m+1 \quad \forall k \in K \quad (2)$$

$$\text{circuit}(\text{Succ}) \quad (3)$$

$$\text{Time}_v + s_v + d_{v, \text{Succ}_v} \leq \text{Time}_{\text{Succ}_v} \quad \forall v \in V \setminus V_{\text{end}} \quad (4)$$

$$e_v \leq \text{Time}_v \leq l_v \quad \forall v \in V \quad (5)$$

$$\text{Load}_{\alpha_k} = 0 \quad \forall k \in K \quad (6)$$

$$\text{Load}_v + q_v = \text{Load}_{\text{Succ}_v} \quad \forall v \in V \setminus V_{\text{end}} \quad (7)$$

$$0 \leq \text{Load}_v \leq c \quad \forall v \in V \quad (8)$$

$$\text{Vehicle}_v = \text{Vehicle}_{\text{Succ}_v} \quad \forall v \in V \setminus V_{\text{end}} \quad (9)$$

$$\text{Vehicle}_{r_i^+} = \text{Vehicle}_{r_i^-} \quad \forall r_i \in R \quad (10)$$

$$\text{Time}_{r_i^-} - (\text{Time}_{r_i^+} + s_{r_i^+}) \leq L \quad \forall r_i \in R \quad (11)$$

$$\text{Time}_{\omega_k} - \text{Time}_{\alpha_k} \leq D \quad \forall k \in K \quad (12)$$

Sequence Variable Model

$$\min \sum_{k \in K} \text{Dist}_k \quad (13)$$

$$\text{Distance}(\text{Route}_k, d, \text{Dist}_k) \quad \forall k \in K \quad (14)$$

$$\sum_{k \in K} \mathcal{R}_v(\text{Route}_k) = 1 \quad \forall v \in V_{\text{tr}} \quad (15)$$

$$\text{first}(\text{Route}_k) = \alpha_k \quad \forall k \in K \quad (16)$$

$$\text{last}(\text{Route}_k) = \omega_k \quad \forall k \in K \quad (17)$$

$$\text{TransitionTimes}(\text{Route}_k, \text{Time}, s, d) \quad \forall k \in K \quad (18)$$

$$e_v \leq \text{Time}_v \leq l_v \quad \forall v \in V \quad (19)$$

$$\text{Cumulative}(\text{Route}_k, r^+, r^-, q, c) \quad \forall k \in K \quad (20)$$

$$\text{Time}_{r_i^-} - (\text{Time}_{r_i^+} + s_{r_i^+}) \leq L \quad \forall r_i \in R \quad (21)$$

$$\text{Time}_{\omega_k} - \text{Time}_{\alpha_k} \leq D \quad \forall k \in K \quad (22)$$

The search. In Constraint Programming (CP), finding a solution typically relies on a backtracking depth-first search (DFS). This search is customized by a branching procedure that selects a variable and assigns it a value at each step. A branching procedure is also often referred to as a heuristic. In particular, the leftmost branch of the search tree should mimic a strong greedy construction heuristic in order to quickly find a high-quality first solution.

In the classical successor model, branching typically consists of selecting a node v without assigned successor Succ_v , often following a *first-fail* principle by choosing the node with the smallest Succ_v domain size, and trying different values for its successor. By evaluating values in order of increasing distance, the leftmost branch naturally mimics a *nearest neighbor* strategy, building routes forward by repeatedly appending the closest unvisited node. This standard procedure is illustrated in Algorithm 1. However, as shown by (Rosenkrantz et al. 1977), nearest neighbor strategies often suffer from the “long edge effect,” where a very long and costly connection is required to close the tour.

Algorithm 1: Creation of the branching points in the Successor Model.

```

1 if  $\bigwedge_{v \in V} \text{Succ}_v.\text{isFixed}()$  then
2   | return solution
3  $v \leftarrow \underset{u \in V | \neg \text{Succ}_u.\text{isFixed}()}{\text{argmin}} |\text{Succ}_u.\text{domain}()|$ 
4  $\text{branches} \leftarrow \{\}$ 
5 for  $w \in \text{Succ}_v.\text{domain}()$  do
6   |  $\text{branches} \leftarrow \text{branches} \cup \{\text{Succ}_v = w\}$ 
7 sort  $\text{branches}$  by increasing distance  $d_{v,w}$ 
8 return  $\text{branches}$ 

```

Conversely, an *insertion-based* heuristic avoids this issue by selecting an unvisited node and inserting it into the position that minimizes the cost increase within a partial route. While an insertion-based branching strategy cannot be easily achieved in a successor model, the sequence variable model natively supports it. Because sequence variables explicitly represent the ordered route, the branching decision directly corresponds to choosing an uninserted node and inserting it at a specific position in the sequence. By sorting these insertion decisions by their cost increase, the leftmost branch exactly mimics a greedy insertion heuristic.

An insertion-based branching procedure for the sequence variable model is presented in Algorithm 2. This procedure generates the alternative child nodes of the search tree by considering all feasible insertions of a chosen unvisited node into the sequence variables representing the active routes.

Algorithm 2: Creation of the branching points in the Sequence Variable Model.

```

1 if  $\bigwedge_{k \in K} \text{Route}_k.\text{isFixed}()$  then
2   | return solution
3  $v \leftarrow \underset{u \in V | u \text{ not yet inserted}}{\operatorname{argmin}} \sum_{k \in K} \text{Route}_k.\text{nInsert}(u)$ 
4  $\text{branches} \leftarrow \{\}$ 
5 for  $k \in K$  do
6   |  $I \leftarrow \text{Route}_k.\text{getInsert}(v)$ 
7   | for  $p \in I$  do
8   | |  $\text{branches} \leftarrow \text{branches} \cup \{\text{Route}_k.\text{insert}(p, v)\}$ 
9 sort  $\text{branches}$  by increasing heuristic value
10 return  $\text{branches}$ 

```

The branching proceeds in two steps. First it prioritizes the unvisited node v with the fewest remaining feasible insertions (line 3). This is in line with the *first-fail* principle, which aims to minimize the depth of the search tree by making decisions that are most likely to lead to an early failure if no solution exists in the current sub-tree. Given this node, all valid insertion points I in a given sequence variable Route_k are retrieved (line 6). Each feasible predecessor p is considered as an alternative decision to explore (line 8). Since DFS is used, the most promising insertions should be explored first. This can, for instance, be achieved by sorting all candidate insertions based on their impact on tour length, prioritizing those that minimize the increase in distance (line 9). A solution is reached when no further insertions are possible in any vehicle: all paths are fixed (line 2).

A DFS using the branching of Algorithm 2 can be used to find an initial solution by stopping at the first feasible solution encountered. A limited-discrepancy DFS search, keeping only the leftmost few branches, as was done in (Shaw 1998), can also be used to quickly discover good solutions.

Large Neighborhood Search. These strategies can also be embedded into a Large Neighborhood Search (LNS) framework (Shaw 1998). Intuitively, LNS iteratively improves a solution by destroying a part of it (relaxing a subset of variables) and then reconstructing it using a search procedure. In the context of vehicle routing, this typically means removing a set of customer visits from the current routes and then re-inserting them at cheaper positions. Because LNS relies on a sequence of local, restricted searches rather than an exhaustive exploration of the full search space, it is a highly effective heuristic to quickly discover good solutions, though it provides no guarantee of finding the optimal one.

When applying LNS to the classical successor model, relaxing a set of requests means unbinding the *Succ* variables associated with their pickup and delivery nodes, as well as the *Succ* variables of the nodes that preceded them in the previous solution. As shown in Algorithm 3, this process breaks the sequence into multiple

disconnected segments. The CP solver must then re-link these segments during the search phase while inserting the relaxed nodes.

Algorithm 3: A LNS iteration for DARP in the Successor Model

Input: S : a successor array representing a visit mapping for each vehicle

```

1  $\mathcal{R} \leftarrow \text{relaxedRequests}(S)$ 
2  $\mathcal{V}^{\mathcal{R}} \leftarrow \bigcup_{r_i \in \mathcal{R}} \{r_i^+, r_i^-\}$ 
3 for  $v \in V$  do
4    $w \leftarrow S[v]$  // The successor of  $v$  in the previous solution
5   if  $v \notin \mathcal{V}^{\mathcal{R}}$  and  $w \notin \mathcal{V}^{\mathcal{R}}$  then
6      $\text{Succ}_v \leftarrow w$  // Keep link if both nodes are kept
7   else
8      $\text{Succ}_v \leftarrow \text{unbound}$  // Relax connection
9  $S \leftarrow \text{Solve problem starting with partial Succ}$ 

```

In contrast, applying LNS to the sequence variable model naturally maintains the partial tours, exactly corresponding to the original LNS for vehicle routing presented by (Shaw 1998). As depicted in Algorithm 4, the nodes belonging to relaxed requests are simply omitted when reconstructing the partial sequence (line 7), while all remaining nodes seamlessly keep their relative ordering and adjacent connections. A search is then performed (line 8) to insert the relaxed requests back into these continuous partial paths, leading to a new solution $\hat{S}$. This process is repeated until a given stopping criterion is met.

Algorithm 4: A LNS iteration for DARP in the Sequence Variable Model

Input: $S = [S_1, \dots, S_K]$: a sequence of visits for each vehicle

```

1  $\mathcal{R} \leftarrow \text{relaxedRequests}(S)$ 
2  $\mathcal{V}^{\mathcal{R}} \leftarrow \bigcup_{r_i \in \mathcal{R}} \{r_i^+, r_i^-\}$ 
3 for  $k \in K$  do
4    $\text{Route}_k \leftarrow \text{empty sequence variable}$  // empty the vehicle path
5   for  $v \in S_k$  do // iterate over the previous path  $S_k$ , in order
6     if  $v \notin \mathcal{V}^{\mathcal{R}}$  then
7        $\text{Route}_k.\text{insertAtEnd}(v)$  // append the visit in same order as in  $S$ 
8  $S \leftarrow \text{Solve problem starting with Route}$  // Using branching from Algorithm 6

```

The structural difference between Algorithm 3 and Algorithm 4 illustrates how sequence variables natively support local search heuristics like the *relocate* move (Savelsbergh 1985). In a sequence variable model, relaxing a node v maintains the relative ordering of the remaining nodes, directly corresponding to removing v from the route. The solver then evaluates its re-insertion at all feasible positions. Conversely, emulating this move in a successor model requires unbinding the predecessors and successors of both v and its target insertion position. Because the optimal target is unknown a priori, one must relax the successors of all potential targets, which effectively destroys the route structure and unnecessarily expands the search space. By preserving partial routes, sequence variables natively generalize the relocate move to multi-node re-insertions, exploring neighborhoods analogous to k -opt or exchange moves efficiently within a single search tree.

2 Limitations of Existing Approaches in CP

The sequence variable model and search strategy introduced for the DARP in the previous section rely on insertion-based sequence variables. The successor model, also presented in the previous section and widely used in CP, does not support insertion-based search strategies. A successor model uses an integer variable $succ_i$ for representing the direct successor of customer i in a tour. To ensure that all customers are visited on a tour, the (Hamiltonian) *circuit* global constraint (Lauriere 1978) and its weighted variants (Benchimol et al. 2012) can be used. This kind of model has two main limitations. First, representing the optional nature of visits with the successor model is not straightforward¹. Second, at the search level, the first solution along the leftmost branch of the search tree typically relies on a *nearest neighbor* heuristic. These nearest neighbor heuristics tend to add small edges near the root of the search tree, but as decisions progress, they tend to add very long edges at the end, making it difficult to quickly find good solutions with CP.

An advanced CP alternative to the successor model, which enables dealing with optional visits more naturally, is to use the *head-tail sequence variables* implemented in IBM CP Optimizer (Laborie et al. 2009, 2018). According to the documentation and in the context of VRP, the domain consists of two growing sequences of nodes. One is the head (prefix) and the other the tail (suffix) of nodes to be visited in that order. The domain update operations append a node to the tail, append one to the head, or merge the two to form the final sequence. These variables handle optionality by design since not every node needs to be added to the sequence. However, similarly to the successor model, a heuristic on those variables would also rely on a nearest neighbor strategy.

Since both the successor and the head-tail sequence models do not easily support insertion heuristics, a new type of variable, the insertion-based sequence variable, was recently introduced in (Delecluse et al. 2022; Thomas et al. 2020) to address these limitations. These variables have a domain composed of the possible insertions for each node within a partial sequence. They thus consume more space than the head-tail sequence variables. A summary of the main properties of the modeling variables for VRP in CP is given in Table 1.

Table 1. Characteristics of CP variables.

Feature	Type of variable		
	Successor	Head-tail sequence	Insertion-based sequence
Nearest neighbor heuristics	✓	✓	✓
Insertion-based heuristics			✓
Optional visits		✓	✓
Memory complexity on VRP with n nodes and k vehicles	$O(n^2)$	$O(k \cdot n)$	$O(k \cdot n^2)$

In the rest of the article, insertion-based sequence variables are simply denoted *sequence variables*. This work significantly extends the previous work on sequence variables from (Delecluse et al. 2022; Thomas et al. 2020) by:

- **Formalizing the computational model for sequence variables**, providing a robust theoretical foundation.
- **Introducing consistency levels**, including the novel concept of *insert consistency*.
- **Proposing an implementation** along with the underlying data structures to integrate sequence variables into existing trail-based CP solvers.

¹Some modeling languages (Boussemart, Lecoutre, et al. 2016; Hentenryck 2002; Nethercote et al. 2007) offer a subcircuit version that allows a node to be excluded from the circuit by assigning it a self-loop ($succ_i = i$) but this complicates the model semantics, as constraints need to be aware that self-loops are permitted and designate an unvisited node

- **Developing global constraints** for modeling VRP with sequence variables.
- **Demonstrating the effectiveness of sequence variables** on several vehicle routing problems and on one sequencing problem.

This paper is organized as follows. Section 2 discusses the limitations of existing CP approaches for vehicle routing. Section 3 provides a brief overview of VRP solving with CP and reviews work related to sequence variables. Section 4 details the computational model for sequence variables, including domain representations, operations, and consistency definitions. Section 7 discusses global constraints tailored for sequence variables and their integration with vehicle routing problems. Section 8 describes search strategies to efficiently solve CP models involving sequence variables. Finally, Section 9 presents experimental results on the Dial-A-Ride Problem, its variants without some or all of the temporal constraints, and on one sequencing problem. Those results validate the theoretical contributions and practical implementations.

3 Related Work

Constraint Programming (CP) is a declarative paradigm where problems are modeled through variables and constraints. Solutions are then found via a systematic search combined with constraint propagation. In CP, each variable x has an associated domain $D(x)$ of possible values, and constraints actively filter these domains by removing inconsistent values. At each node of the search tree, the search selects a variable and a value from its domain, applies a branching decision—typically $x = v$ or $x \neq v$ —and propagates the constraints until a fix-point is reached. It backtracks if an inconsistency is detected. We refer to (Michel et al. 2021) for a more detailed background on CP solving.

The successor model described in (Kilby and Shaw 2006) is commonly used for modeling VRPs in CP, where each node is associated with an integer variable representing its next visited node. An attempt to extend this model to support insertion-based search was proposed in (Kilby, Prosser, et al. 2000), where a reversible data structure called *insertion schedule* represents the direct predecessor and successor of the nodes already inserted by the branching procedure. A channeling mechanism is then used to maintain consistency with a new set of variables, called the *insertion variables* π , which represent the possible insertion positions for visit i . Although this approach enables advanced insertion procedures, such as ours, it does not provide a well-defined sequence variable with a computation domain that can be exploited to filter external constraints.

Another CP alternative relies on dedicated structured domains. They have been successfully introduced for representing set variables in (Gervet 1997; Puget 1993) and graph variables in (Dooms et al. 2005). These two domains rely on the concept of subset-bounds, which maintain a lower bound of mandatory elements and an upper bound of potential elements in the set or graph. In (Pesant et al. 1997), the authors propose an extension of the successor model by introducing, for each visit i , two sets: $\mathcal{A}_i$ and $\mathcal{B}_i$, representing respectively the set of (not necessarily direct) predecessors and successors in the tour. These sets are maintained through channeling constraints with the successor and predecessor variables. This formulation makes it possible to add precedence constraints to the model and, by introducing them dynamically, one can also implement insertion-based search strategies. However, as with the *insertion variables* π of (Kilby, Prosser, et al. 2000), this approach does not provide a well-defined sequence variable with a computation domain that can be directly exploited by external constraints.

The closest related work to the insertion-based sequence variable domain presented in this paper is the head-tail sequence variable explained in the previous section and introduced in (Laborie et al. 2009, 2018). A similar approach is implemented in OR-Tools by Google (Cuvelier et al. 2023; Perron and Furnon 2019), which also provides sequence variable modeling. While primarily targeted at scheduling problems, these sequence variables have been successfully used to solve hybrid routing-and-scheduling problems in works such as (Cappart et al.

2018; Liu et al. 2018). A related idea of growing prefix sequences was utilized for path variable representation in traffic engineering problems in the field of computer networks (Hartert et al. 2015).

The approach proposed in this paper is more flexible, allowing elements to be inserted at arbitrary positions of the sequence. This insertion capability can be viewed as a domain implementation of the insertion graph concept introduced in (Bent and Van Hentenryck 2004). This domain representation was previously introduced in (Thomas et al. 2020). A simplified variant without a set of required nodes is detailed in (Delecluse et al. 2022). This paper builds upon and refines these earlier works, providing a cleaner and more detailed version. It formally defines the computation domain and its semantics, and introduces the notion of consistency.

4 Sequence Variable

We first introduce some notation before defining the domain of sequence variables.

Notations. A sequence $\vec{s}$ is defined as an ordered set of nodes belonging to a graph, without repetition. Let $\vec{s}$ be a sequence with the form $\vec{s} = \vec{s}_1 \cdot v_1 \cdot \vec{s}_2$ ($\vec{s}_1$ and $\vec{s}_2$ being sequences, possibly empty). An insertion operation $\xrightarrow{(v_1, v_2)}$ defined by the pair of nodes (v_1, v_2) produces a new sequence $\vec{s}' = \vec{s}_1 \cdot v_1 \cdot v_2 \cdot \vec{s}_2$. We denote with $\vec{s} \subset \vec{s}'$ that the sequence $\vec{s}$ is a subsequence of $\vec{s}'$, where $\vec{s}'$ preserves the order from $\vec{s}$ and has strictly more nodes. Then $\vec{s}'$ is called a supersequence of $\vec{s}$. If the two sequences may be the same, the relation is written $\vec{s} \subseteq \vec{s}'$. We denote $v_i \xrightarrow{\vec{s}} v_j$ to indicate that v_i directly precedes v_j in the sequence $\vec{s}$ and $v_i \prec v_j$ when v_i precedes (not necessarily directly) v_j in $\vec{s}$. Those relations are simply written $v_i \rightarrow v_j$ and $v_i \prec v_j$ when clear from the context. If the nodes may be the same, the relation is written $v_i \preceq v_j$. Given a sequence $\vec{s} = v_1 \dots v_i \dots v_n$, $\text{prefix}(\vec{s}, v_i) = v_1 \dots v_i$ and $\text{suffix}(\vec{s}, v_i) = v_i \dots v_n$. Lastly, given a node set V , a start node $\alpha \in V$ and an end node $\omega \in V$, $\mathcal{P}(V)$ denotes all sequences $\vec{s}$ over a subset of nodes V , starting at node $\alpha \in V$ and ending at node $\omega \in V$.

Domain. A sequence domain denoted $\mathcal{D} \subseteq \mathcal{P}(V)$ contains a set of sequences over a subset of the nodes V without repetition, starting at node $\alpha \in V$ and ending at node $\omega \in V$. Four elementary restrictions can be imposed on a sequence domain:

- (1) **Require** a node to be visited, without explicitly stating the position of the node.
- (2) **Exclude** a node from being visited.
- (3) **Enforce a subsequence** $\vec{s}'$ to be included (i.e., enforce $\vec{s}' \subseteq \vec{s}$).
- (4) **NotBetween**, which forbids a node v_2 to be placed between two other nodes v_1 and v_3 regardless of any additional nodes that may be present between them.

Those restrictions were chosen to enable branching decisions, to allow for large neighborhood searches, and to facilitate domain filtering through constraints.

To define the sequence domain $\mathcal{D}$, we first define one subdomain for each of the four possible restrictions:

- (1) $\mathcal{D}^{\oplus}(R)$ is specified by a set of required nodes $R \subseteq V$. It denotes all sequences including all nodes in R . More formally,

$$\mathcal{D}^{\oplus}(R) = \left\{ \vec{s} \in \mathcal{P}(V) \mid \forall v \in R : v \in \vec{s} \right\} \quad (23)$$

- (2) $\mathcal{D}^{\otimes}(X)$ is specified by a set of excluded nodes $X \subseteq V$. It denotes all sequences where no node in X is included. More formally,

$$\mathcal{D}^{\otimes}(X) = \left\{ \vec{s} \in \mathcal{P}(V) \mid \forall v \in X : v \notin \vec{s} \right\} \quad (24)$$

- (3) $\mathcal{D}^{\textcircled{S}}(\vec{s})$ is specified by a partial sequence of nodes $\vec{s}$. It denotes all sequences $\vec{s}$ such that $\vec{s}$ is a subsequence of $\vec{s}$. More formally,

$$\mathcal{D}^{\textcircled{S}}(\vec{s}) = \left\{ \vec{s} \in \mathcal{P}(V) \mid \vec{s} \subseteq \vec{s} \right\} \quad (25)$$

- (4) $\mathcal{D}^{\textcircled{N}}(N)$ is specified by a set of NotBetween triples $N \subseteq V \times V \times V$. $\mathcal{D}^{\textcircled{N}}(N)$ denotes all sequences in which no NotBetween triple from N appears. More formally,

$$\mathcal{D}^{\textcircled{N}}(N) = \left\{ \vec{s} \in \mathcal{P}(V) \mid \forall (v_i \cdot v_j \cdot v_k) \in N : (v_i \cdot v_j \cdot v_k) \not\subseteq \vec{s} \right\} \quad (26)$$

The intersection of those four subdomains defines the sequence domain.

Definition 4.1. A sequence domain is defined as

$$\mathcal{D}(R, X, \vec{s}, N) = \mathcal{D}^{\textcircled{R}}(R) \cap \mathcal{D}^{\textcircled{X}}(X) \cap \mathcal{D}^{\textcircled{S}}(\vec{s}) \cap \mathcal{D}^{\textcircled{N}}(N) \quad (27)$$

Definition 4.2. $\mathcal{D}(R, X, \vec{s}, N)$ is said to be fixed when $|\mathcal{D}(R, X, \vec{s}, N)| = 1$.

By abuse of notation, $\mathcal{D}(R, X, \vec{s}, N)$ will be simply written $\mathcal{D}$ in the following.

Example 4.3. $V = \{\alpha, v_1, v_2, v_3, \omega\}$, $R = \{\alpha, \omega, v_2\}$, $X = \{v_3\}$, $\vec{s} = \alpha \cdot v_1 \cdot \omega$, $N = \{(\alpha \cdot v_2 \cdot v_1), (v_3 \cdot v_1 \cdot v_2)\}$. $\mathcal{D} = \mathcal{D}^{\textcircled{R}}(R) \cap \mathcal{D}^{\textcircled{X}}(X) \cap \mathcal{D}^{\textcircled{S}}(\vec{s}) \cap \mathcal{D}^{\textcircled{N}}(N) = \{(\alpha \cdot v_1 \cdot v_2 \cdot \omega)\}$. The compositions of the subdomains are represented in Table 2. Given that $|\mathcal{D}| = 1$, the domain is fixed.

Table 2. The compositions of subdomains in Example 4.3. A check mark in one of the last 4 columns indicates that the corresponding sequence in the first column is included within the subdomain.

$\mathcal{P}(V)$	$\mathcal{D}^{\textcircled{R}}(R)$	$\mathcal{D}^{\textcircled{X}}(X)$	$\mathcal{D}^{\textcircled{S}}(\vec{s})$	$\mathcal{D}^{\textcircled{N}}(N)$	$\mathcal{P}(V)$	$\mathcal{D}^{\textcircled{R}}(R)$	$\mathcal{D}^{\textcircled{X}}(X)$	$\mathcal{D}^{\textcircled{S}}(\vec{s})$	$\mathcal{D}^{\textcircled{N}}(N)$
$\alpha \cdot \omega$		✓		✓	$\alpha \cdot v_3 \cdot v_1 \cdot \omega$			✓	✓
$\alpha \cdot v_1 \cdot \omega$		✓	✓	✓	$\alpha \cdot v_3 \cdot v_2 \cdot \omega$	✓			✓
$\alpha \cdot v_2 \cdot \omega$	✓	✓		✓	$\alpha \cdot v_1 \cdot v_2 \cdot v_3 \cdot \omega$	✓		✓	✓
$\alpha \cdot v_3 \cdot \omega$				✓	$\alpha \cdot v_1 \cdot v_3 \cdot v_2 \cdot \omega$	✓		✓	✓
$\alpha \cdot v_1 \cdot v_2 \cdot \omega$	✓	✓	✓	✓	$\alpha \cdot v_2 \cdot v_1 \cdot v_3 \cdot \omega$	✓		✓	
$\alpha \cdot v_1 \cdot v_3 \cdot \omega$			✓	✓	$\alpha \cdot v_2 \cdot v_3 \cdot v_1 \cdot \omega$	✓		✓	
$\alpha \cdot v_2 \cdot v_1 \cdot \omega$	✓	✓	✓		$\alpha \cdot v_3 \cdot v_1 \cdot v_2 \cdot \omega$	✓		✓	
$\alpha \cdot v_2 \cdot v_3 \cdot \omega$	✓			✓	$\alpha \cdot v_3 \cdot v_2 \cdot v_1 \cdot \omega$	✓		✓	

Domain updates over $\mathcal{D}$ are defined by growing the sets R , X and N , adding more elements to them, or by inserting a node within the partial sequence $\vec{s}$. Although $\vec{s}$ statically defines the subdomain $\mathcal{D}^{\textcircled{S}}(\vec{s})$, it can only be extended through insertions. Let $v_i \xrightarrow{\vec{s}} v_k$ and $v_j \notin \vec{s}$. Inserting v_j after v_i replaces $v_i \cdot v_k$ by $v_i \cdot v_j \cdot v_k$ in $\vec{s}$, thereby retaining precisely the sequences of the current domain satisfying $v_i \prec v_j \wedge v_j \prec v_k$. This insertion does not fix the final successor of v_j , as later insertions may occur between v_j and v_k . Conversely, $\text{notBetween}(v_i, v_j, v_k)$ retains precisely the sequences satisfying $\neg(v_i \prec v_j \wedge v_j \prec v_k)$, allowing v_j to be inserted elsewhere or to be absent. Thus, as with Require and Exclude, insertion and notBetween are complementary updates associated with the candidate insertion of v_j between v_i and v_k . Notice that elementary restrictions can be combined to enforce a *between* restriction, that is, forcing a node v_2 to be visited between nodes v_1, v_3 . This can be achieved by adding $(\alpha \cdot v_2 \cdot v_1), (v_3 \cdot v_2 \cdot \omega)$ to the set of NotBetween N given that any sequence $\vec{s} \in \mathcal{D}$ begins at α and ends at ω . Moreover, if node v_2 must be included in the sequences of the domain, v_2 can be added to the required nodes R .

Each sequence variable $\vec{s}$ is associated with a sequence domain, and represents an unknown sequence. Such a variable is particularly convenient for modeling a route from an origin node α to a destination node ω in a VRP, where the nodes visited and their ordering represents the path performed by a vehicle.

We introduce a compact encoding of the domain $\mathcal{D}$ in the next section, which only consumes $O(n^2)$ memory, with $n = |V|$, and enables efficient domain updates.

5 An $O(n^2)$ encoding of the sequence variable domain

This section introduces a compact domain representation for the entire domain $\mathcal{D}(R, X, \vec{s}, N)$ as a set of pairs of nodes, requiring only $O(n^2)$ memory, with the limitation that for each NotBetween $(v_1 \cdot v_2 \cdot v_3) \in N$, the two extremities v_1, v_3 are within the partial sequence $\vec{s}$. This representation exploits the representation of the intersection of the subdomains. First, the representation of $\mathcal{D}^{\odot}(\vec{s}) \cap \mathcal{D}^{\oplus}(N)$ is introduced in Section 5.1. It is then extended with the intersection of the excluded nodes subdomain $\mathcal{D}^{\otimes}(X)$ in Section 5.2 and, finally, with the intersection of the required nodes subdomain $\mathcal{D}^{\ominus}(R)$ in Section 5.3.

5.1 Partial Sequence and NotBetween

A naive representation of the set of NotBetween $N \subseteq V \times V \times V$ requires a cubic space complexity. This can be reduced to a quadratic one by representing directly $\mathcal{D}^{\odot}(\vec{s}) \cap \mathcal{D}^{\oplus}(N)$ and by restricting N to contain only NotBetween triples with the extremities belonging to the partial sequence $\vec{s}$ and in the order in which they appear in $\vec{s}$: $N \subseteq \{(v_1 \cdot v_2 \cdot v_3) \mid v_1 \prec_{\vec{s}} v_3\}$. The set of NotBetween with this restriction is written $\hat{N}$. The set $\hat{N}$ can then be translated into a set of pairs of nodes (v_i, v_2) corresponding to *forbidden insertions* of v_2 just after v_i in $\vec{s}$, due to a NotBetween:

$$I^{\oplus}(\vec{s}, \hat{N}) = \{(v_i, v_2) \mid (v_1 \cdot v_2 \cdot v_3) \in \hat{N} \wedge v_1 \preceq v_i \wedge v_i \prec v_3\} \quad (28)$$

THEOREM 5.1. *The forbidden insertions $I^{\oplus}(\vec{s}, \hat{N})$ can be used instead of $\hat{N}$ in the representation of the domain $\mathcal{D}^{\odot}(\vec{s}) \cap \mathcal{D}^{\oplus}(N)$.*

PROOF. One can enumerate all supersequences $\vec{s}'$ of $\vec{s}$ such that no forbidden pair of $I^{\oplus}(\vec{s}, \hat{N})$ appears in that order in $\vec{s}'$:

$$\mathcal{D}^{\odot}(\vec{s}) \cap \mathcal{D}^{\oplus}(\hat{N}) = \{\vec{s}' \mid (\vec{s} \subseteq \vec{s}') \wedge \left(\forall v \in (\vec{s}' \setminus \vec{s}) : (\text{prev}(v, \vec{s}, \vec{s}'), v) \notin I^{\oplus}(\vec{s}, \hat{N}) \right)\} \quad (29)$$

where $\text{prev}(v, \vec{s}, \vec{s}')$ denotes the first node preceding v in $\vec{s}'$ and also appearing in $\vec{s}$:

$$\text{prev}(v, \vec{s}, \vec{s}') = \begin{cases} v & \text{if } v \in \vec{s} \\ \text{prev}(v_i, \vec{s}, \vec{s}') \mid v_i \xrightarrow{\vec{s}'} v & \text{otherwise.} \end{cases} \quad (30)$$

□

Thanks to Theorem 5.1, $\mathcal{D}^{\odot}(\vec{s}) \cap \mathcal{D}^{\oplus}(\hat{N})$ can be represented solely with the partial sequence $\vec{s}$ and the pairs of nodes $I^{\oplus}(\vec{s}, \hat{N})$, computed in $O(n^2)$ time and also consuming $O(n^2)$ in memory.

Domain updates. Two domain updates are possible on $\mathcal{D}^{\odot}(\vec{s}) \cap \mathcal{D}^{\oplus}(\hat{N})$: insertion of a node within $\vec{s}$, or adding a new NotBetween triple to $\hat{N}$. Two possible strategies to update $I^{\oplus}(\vec{s}, \hat{N})$ under these modifications are

- (1) Recompute $I^{\oplus}(\vec{s}, \hat{N})$, which still requires maintaining $\hat{N}$ and requires $O(n^3)$ memory.
- (2) Store the set $I^{\oplus}(\vec{s}, \hat{N})$, consuming $O(n^2)$ in memory, and incrementally update it.

We follow the more efficient second strategy and show how to perform such updates in $O(n)$ time while maintaining an $O(n^2)$ memory bound:

- If a new NotBetween $(v_1 \cdot v_2 \cdot v_3)$ is added to $\hat{N}$, the newly forbidden insertions are:

$$\Delta \mathcal{I}^{\oplus}(\vec{s}, (v_1 \cdot v_2 \cdot v_3)) = \{(v_i, v_2) \mid v_1 \preceq v_i \wedge v_i \prec v_3\} \quad (31)$$

This uses the same rule as (28). At most $O(|\vec{s}|) = O(n)$ pairs can be added in this manner.

- If the sequence $\vec{s}$ is expanded into $\vec{s}'$ through an insertion $\vec{s} \xrightarrow{(v_1, v_2)} \vec{s}'$, the newly forbidden insertions are:

$$\Delta \mathcal{I}^{\oplus}(\vec{s}, (v_1, v_2)) = \{(v_2, v_i) \mid (v_1, v_i) \in \mathcal{I}^{\oplus}(\vec{s}, \hat{N})\} \quad (32)$$

This means that a node v_i that could not be placed directly after v_1 now cannot be placed directly after v_2 either. The set (32) can be created in $O(n)$ time and contains $O(n)$ pairs.

Whenever a NotBetween $(v_1 \cdot v_2 \cdot v_3)$ is added to $\hat{N}$ while it is a subsequence of the partial sequence $((v_1 \cdot v_2 \cdot v_3) \subseteq \vec{s})$, it results in a domain wipeout. Similarly, expanding $\vec{s}$ into $\vec{s}'$ using a forbidden insertion $(v_1, v_2) \in \mathcal{I}^{\oplus}(\vec{s}, \hat{N})$ (i.e., performing $\vec{s} \xrightarrow{(v_1, v_2)} \vec{s}'$) results in a domain wipeout. Next, we show how to compute the intersection with the remaining subdomains.

5.2 Excluded Nodes

The subdomain $\mathcal{D}^{\oplus}(\vec{s}) \cap \mathcal{D}^{\oplus}(\hat{N})$ presented previously only accounted for a partial sequence $\vec{s}$ and a set of NotBetween constraints $\hat{N}$. Excluding a node v from the sequence can be reduced to adding the NotBetween $(\alpha \cdot v \cdot \omega)$ to $\hat{N}$, given that all sequences begin at α and end at ω . Therefore, for a set of excluded nodes X , we have:

$$\forall v \in X : (\alpha \cdot v \cdot \omega) \in \hat{N} \quad (33)$$

When an update $X \leftarrow X \cup \{v\}$ occurs, it suffices to add the NotBetween (α, v, ω) , and use (31) to trigger the incremental update of $\mathcal{I}^{\oplus}(\vec{s}, \hat{N})$. Thus, it is not mandatory to store the excluded nodes X .

5.3 Required Nodes

To take into account a set of required nodes R , we have no option other than to store them. This requires an additional $O(n)$ memory. The entire domain $\mathcal{D}$ can be represented in $O(n^2)$ with i) the set of pairs of nodes $\mathcal{I}^{\oplus}(\vec{s}, \hat{N})$, ii) the partial sequence $\vec{s}$, and iii) the set of required nodes R . Together, those elements implicitly represent all supersequences $\vec{s}'$ of $\vec{s}$ such that no forbidden pair of $\mathcal{I}^{\oplus}(\vec{s}, \hat{N})$ appears in that order in $\vec{s}'$ and all the required nodes are present in $\vec{s}'$:

$$\mathcal{D} = \{\vec{s}' \mid (\vec{s} \subseteq \vec{s}') \wedge \left(\forall v \in (\vec{s}' \setminus \vec{s}) : (\text{prev}(v, \vec{s}, \vec{s}'), v) \notin \mathcal{I}^{\oplus}(\vec{s}, \hat{N}) \right) \wedge (\forall v \in R : v \in \vec{s}')\} \quad (34)$$

The next section proposes an implementation of these elements using a graph data structure.

6 Compact domain implementation

As explained in the previous section, a domain implementation requires representing the forbidden insertions $\mathcal{I}^{\oplus}(\vec{s}, \hat{N})$, the partial sequence $\vec{s}$, and the required nodes R .

Rather than representing $\mathcal{I}^{\oplus}(\vec{s}, \hat{N})$, we instead represent the complementary set $(V \times V) \setminus \mathcal{I}^{\oplus}(\vec{s}, \hat{N}) \setminus \{(v_i, v_j) \mid v_i, v_j \in \vec{s} \wedge v_i \not\prec v_j\}$ as a graph $G(V, E)$. Each directed edge $(v_i, v_j) \in E$ with $v_i \in \vec{s}$ represents a possible insertion of node v_j in $\vec{s}$ just after v_i in the partial sequence $\vec{s}$. Notice that every pair of nodes in the partial sequence that do not directly follow each other are also removed from the set of edges. As the partial sequence $\vec{s}$ grows and the set of NotBetween constraints $\hat{N}$ grows, the set of edges E can only shrink monotonically. Eventually, when the variable is fixed, the set of edges forms a path that corresponds to the partial sequence $\vec{s}$. This complementary view enables the enumeration of all feasible insertions of a node into the partial sequence $\vec{s}$. It also allows for

counting feasible insertions for each node in the partial sequence. These are two common operations used by filtering algorithms and for implementing first-fail branching heuristics.

6.1 A Reversible Data Structure

The main mechanism for restoring the state of domains and constraints on backtracking in a CP solver is called *trailing* (we refer to (Michel et al. 2021) for an introduction to trailing in CP solvers). To be integrated into a standard trail-based CP solver, the compact domain implementation also needs to be *reversible*, meaning that its state is restored when the search backtracks. In CP, this is typically achieved using a *trail*, a data structure that records changes to variables. A *reversible integer* is an integer whose modifications are automatically recorded on the trail for restoration. The graph $G(V, E)$ is implemented as an adjacency list where each list is a reversible set of nodes and the partial sequence $\vec{s}$ uses a reversible successor array. In more detail, the implementation relies on the following reversible data structures, most of which were already present in (Delecluse et al. 2022; Thomas et al. 2020):

- The partial sequence $\vec{s}$ is encoded using a successor array of reversible integers s^+ . It stores a pointer to the current successor of each node belonging to the partial sequence $\vec{s}$, and an element without a successor (i.e., a node $v \in V \setminus \vec{s}$) points towards itself (self-loop). Similarly, an additional array of reversible integers s^- tracks the current predecessors of each node.
- The set of edges E is maintained by two adjacency sets per node $v \in V$: E_v^- (incoming) and E_v^+ (outgoing) edges: $E_v^- = \{(v_i, v) \mid v_i \in V, (v_i, v) \in E\}$, $E_v^+ = \{(v, v_i) \mid v_i \in V, (v, v_i) \in E\}$. Those sets are implemented using the reversible sparse-sets introduced in (Saint-Marcq et al. 2013), allowing deletion and state restoration in constant time in a trail-based solver.
- A set I , tracking the insertable nodes $\{v \in V \mid v \notin \vec{s} \wedge v \notin X\}$, is represented as a reversible sparse set.
- The size of the partial sequence $\vec{s}$ is tracked by a reversible integer nS .
- Each $v_j \in V$ has a counter nI_j tracking insertions, maintained under domain changes. Heuristics can use it, for example, to select the node with the fewest insertion points.
- Given that R and X are disjoint subsets of V , we use a sparse-set with two reversible size markers as introduced in (Saint-Marcq et al. 2013), ensuring removal of nodes and state restoration in constant time, and enabling iteration over R, X in $O(|R|), O(|X|)$, respectively. We also define $P = V \setminus X \setminus R$ as the set of *possible* nodes, which are neither required nor excluded.

Those datastructures are illustrated in Figure 1.

Invariants on the domain are presented in the Appendix A and the implementation of the update operations in Appendix B. Some coherence is enforced on the domain representation. For instance, given that all nodes in the partial sequence will always be visited, we enforce $\vec{s} \subseteq R$. We also exploit the insertion counter nI_j of node $v_j \in V$ to automatically insert a node when it is required and has only one insertion remaining (similarly to (Kilby, Prosser, et al. 2000)), or to exclude v_j from the sequence when no insertion remains for it. This process is also detailed in Appendix B.

6.2 API

A sequence domain is fixed whenever, for each node that does not belong to the partial sequence, no insertion remains. The worst-case time complexity for constructing a sequence is thus $O(|E|)$. In the worst case, all edges are removed except those that form the sequence.

Table 3 lists all query operations in a sequence variable domain, along with their associated time complexities. Domain updates are described in Table 4.

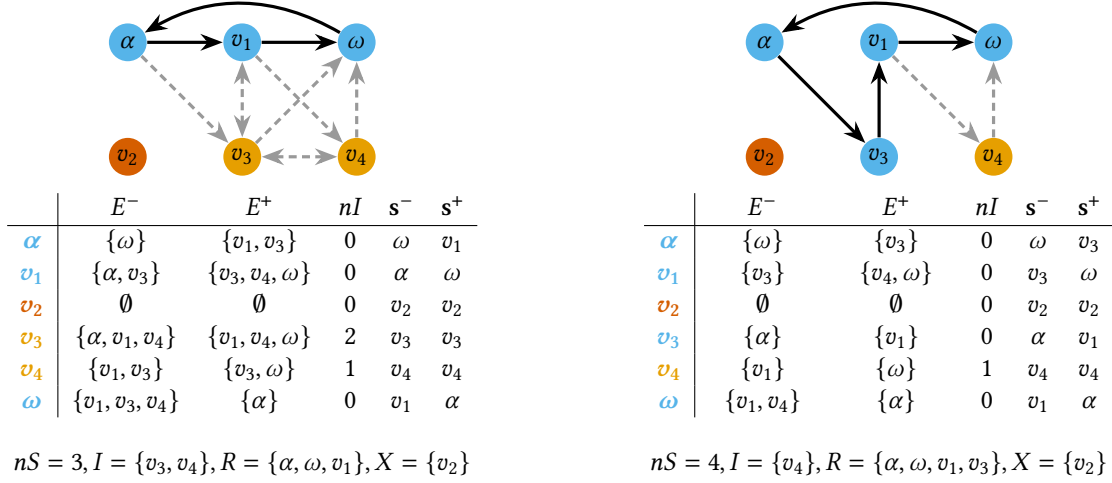


Fig. 1. Compact domain implementation. On the left, the partial sequence $\vec{s}$ and the graph $G(V, E)$ are shown. Below them is a table showing the edges E^- , E^+ , the insertion counters nI , and the successors s^+ and predecessors s^- of the nodes (only relevant for nodes $v \in \vec{s}$). The right-hand part shows the domain after performing an insertion with (α, v_3) , extending the partial sequence.

6.3 Note on Monotonicity of the Domain

The domain of a sequence variable, as defined in Section 4, is partially ordered by the inclusion of its components (partial sequence $\vec{s}$, forbidden insertions $\hat{N}$, and required/excluded sets R, X). A domain $\mathcal{D}_1$ is considered "smaller" than or equal to $\mathcal{D}_2$ if it is more restricted: $\vec{s}_1$ contains $\vec{s}_2$ as a subsequence, $\hat{N}_1 \supseteq \hat{N}_2$, $R_1 \supseteq R_2$, and $X_1 \supseteq X_2$. This partial ordering, combined with the fact that domains can only shrink during search, ensures that the domain representation forms a lattice.

In CP, the propagation process iteratively applies filtering algorithms until no further domain reductions are possible, reaching a *fix-point*. For this fix-point to be unique and independent of the execution order of the propagators, the filtering algorithms must be *monotonic*: if $\mathcal{D}_1 \subseteq \mathcal{D}_2$, then $f(\mathcal{D}_1) \subseteq f(\mathcal{D}_2)$ for any propagator f . All domain update operations provided by our API (Table 4) are monotonic. Furthermore, the internal coherence rules—such as automatically inserting a required node when only one insertion remains—are also monotonic. Provided that the global constraints implemented over sequence variables are monotonic, the propagation will always converge to a unique greatest fix-point.

6.4 Visit of nodes as boolean variables

Given the set of mandatory nodes R , a binary variable $\mathcal{R}_i(\vec{s})$ can be associated with each node $v_i \in V$. It indicates whether v_i is visited by a sequence variable $\vec{s}$ with a compact domain: fixing it to 1 requires v_i , whereas fixing it to 0 excludes v_i . In CP, this can be implemented as a *view* over the compact sequence domain, making the use of such variables inexpensive once a sequence variable has been created, as in (Michel et al. 2021; Schulte and Tack 2013; Van Hentenryck and Michel 2014). The correspondence between operations on those Boolean variables and a sequence domain is presented in Table 5. Fixing a Boolean variable $\mathcal{R}_i(\vec{s})$ to 1 may automatically insert the node v_i into the partial sequence if only one insertion point remains for it, as explained in Appendix B.

Since node membership in a sequence variable is represented by Boolean variables, standard logical constraints can be imposed directly on sequence variables. For instance, $\sum_{v_i \in V} \mathcal{R}_i(\vec{s}) \geq n$ ensures that $\vec{s}$ visits at least

Table 3. Queries on a compact sequence domain.

Operation	Description	Complexity
isFixed()	Returns true if there are no remaining insertions	$O(1)$
isMember(v_i)	Returns true if $v_i \in \vec{s}$	$O(1)$
isRequired(v_i)	Returns true if the node v_i is required	$O(1)$
isExcluded(v_i)	Returns true if the node v_i is excluded	$O(1)$
isPossible(v_i)	Returns true if the node v_i is possible	$O(1)$
isInsertable(v_i)	Returns true if the node v_i is insertable	$O(1)$
getNext(v_i)	Returns the successor s_i^+ of node v_i	$O(1)$
getPrev(v_i)	Returns the predecessor s_i^- of node v_i	$O(1)$
nInsert(v_i)	Returns the number of feasible insertions for v_i	$O(1)$
nMember()	Returns the length of the partial sequence	$O(1)$
getMember()	Enumerates nodes in the partial sequence $\vec{s}$	$\Theta(\vec{s})$
getRequired()	Enumerates the required nodes R	$\Theta(R)$
getExcluded()	Enumerates the excluded nodes X	$\Theta(X)$
getPossible()	Enumerates the possible nodes P	$\Theta(P)$
getInsertable()	Enumerates the insertable nodes I	$\Theta(I)$
getEdgesTo(v_i)	Enumerates E_i^-	$\Theta(E_i^-)$
getEdgesFrom(v_i)	Enumerates E_i^+	$\Theta(E_i^+)$
canInsert(v_i, v_j)	Returns true if (v_i, v_j) is a feasible insertion	$O(1)$
getInsert(v_j)	Enumerates the insertions of v_j : $\{v_i \mid v_i \in E_j^- \wedge \text{canInsert}(v_i, v_j)\}$	$\Theta(\min(\vec{s} , E_j^-))$
getInsertAfter(v_j, p)	Enumerates the insertions of v_j after p : $\{v_i \mid p \prec v_i \wedge \text{canInsert}(v_i, v_j)\}$	$O(\vec{s})$

Table 4. Updates on a compact sequence domain.

Operation	Update	Precondition	Complexity
notBetween(v_i, v_j, v_k)	$N \leftarrow N \cup (v_i \cdot v_j \cdot v_k)$	$v_i, v_k \in \vec{s}$	$\begin{cases} O(I) & \text{if } v_i \rightarrow v_k \\ O(n) & \text{otherwise} \end{cases}$
insert(v_i, v_j)	$\xrightarrow{(v_i, v_j)}$	$v_i \in \vec{s}$	$\Theta(E_j^-)$
insertAtEnd(v_i)	$\xrightarrow{(\text{getPrev}(\omega), v_i)}$		$\Theta(E_i^-)$
require(v_i)	$R \leftarrow R \cup \{v_i\}$		$O(E_i^-)$
exclude(v_i)	$X \leftarrow X \cup \{v_i\}$		$O(E_i^-)$

n nodes, whereas $\mathcal{R}_i(\vec{s}) = \mathcal{R}_j(\vec{s})$ ensures that nodes v_i and v_j are either both visited or both excluded. The following section introduces more complex constraints with dedicated propagators.

7 Global Constraints

Many useful global constraints and their filtering algorithms can be defined on sequence variables. To limit the scope of this paper, we focus on those that are required by a wide range of Vehicle Routing Problem (VRP) applications, such as the Dial-a-Ride Problem. Properly characterizing the amount of filtering that can be achieved

Table 5. Correspondence between operations on a boolean variable $\mathcal{R}_i(Sq)$ defined over a node v_i in a sequence variable $\vec{s}$. The left part shows queries over the domains, while the right part shows updates of the domains.

Boolean variable	Sequence variable	Boolean variable	Sequence variable
$ \mathcal{D}(\mathcal{R}_i(\vec{s})) = 1$	$\iff \neg \vec{s}.\text{isPossible}(v_i)$	$\mathcal{D}(\mathcal{R}_i(\vec{s})) \leftarrow \{true\}$	$\iff \vec{s}.\text{require}(v_i)$
$false \in \mathcal{D}(\mathcal{R}_i(\vec{s}))$	$\iff \neg \vec{s}.\text{isRequired}(v_i)$	$\mathcal{D}(\mathcal{R}_i(\vec{s})) \leftarrow \{false\}$	$\iff \vec{s}.\text{exclude}(v_i)$
$true \in \mathcal{D}(\mathcal{R}_i(\vec{s}))$	$\iff \neg \vec{s}.\text{isExcluded}(v_i)$		

is of great theoretical interest. Similar to the notions of bound-consistency and domain consistency for global constraints involving integer variables (see (Michel et al. 2021)), various consistency levels can be defined for sequence variables.

Consistency of a constraint on a sequence variable.

Definition 7.1. A constraint C over a sequence domain $\mathcal{D}(R, X, \vec{s}, N)$ is *insert consistent*, if and only if for every remaining insertion (v_1, v_2) we have $\mathcal{D}(R, X, \vec{s}', N) \cap C \neq \emptyset$ where $\vec{s} \xrightarrow{(v_1, v_2)} \vec{s}'$.

Definition 7.2. Given a sequence domain $\mathcal{D}(R, X, \vec{s}, N)$, we define its relaxed sequence domain as $\mathcal{D}(\{v \mid v \in \vec{s}\}, X, \vec{s}, N)$, which considers that only nodes in the partial sequence $\vec{s}$ are required.

Definition 7.3. A constraint C over a sequence domain $\mathcal{D}(R, X, \vec{s}, N)$ is *relaxed insert consistent*, if and only if for every remaining insertion (v_1, v_2) on $\mathcal{D}$ we have $\mathcal{D}(\{v \mid v \in \vec{s}'\}, X, \vec{s}', N) \cap C \neq \emptyset$ where $\vec{s} \xrightarrow{(v_1, v_2)} \vec{s}'$.

Given that $\{v \mid v \in \vec{s}\} \subseteq R$, the *relaxed-insert consistency* property for a constraint C is thus a relaxed form of *insert consistency*. The filtering algorithms that we introduce are relatively basic and aim to reach *relaxed-insert consistency* rather than *insert consistency*, which may be NP-hard to reach in polynomial time for some constraints.

7.1 Distance

The Distance constraint is used to represent the travel length in a sequence of nodes. It links an integer variable **Dist** to the traveled distance between nodes in a sequence variable, through transitions defined in a matrix $\mathbf{d} \in \mathbb{Z}^{|V| \times |V|}$ satisfying the triangular inequality (35).

$$\text{Distance}(\vec{s}, \mathbf{d}, \mathbf{Dist}) \leftrightarrow \sum_{\substack{\vec{s} \\ v_i \rightarrow v_j}} d_{i,j} = \mathbf{Dist} \quad (35)$$

Filtering. The filtering is presented in Algorithm 5. First, it computes the traveled distance over the partial sequence $\vec{s}$ (line 1). If the sequence variable is fixed, this fixes the value of the integer variable **Dist** (line 3). Otherwise, this is used to set its lower bound (line 5) and compute the length of the largest insertion still possible (line 6). All insertions for every insertable node $v_j \in I$ are then examined, and if the cost for inserting a node v_j between two consecutive nodes v_i and v_k is too high, the insertion is removed (lines 10 to 12). The use of the `notBetween` operation here crucially relies on the triangle inequality. Since $d_{i,j} + d_{j,k} \geq d_{i,k}$, any insertion (detour) can only increase the total distance. Without this property, removing an insertion because it currently exceeds the maximum distance would be invalid, as adding even more nodes later in a longer subsequence could theoretically result in a shorter overall path. In the worst case, the filtering empties all the edges from E , except those in the current sequence $\vec{s}$, resulting in a time complexity of $O(|E|)$.

Algorithm 5: Distance($\vec{S}, \mathbf{d}, \mathbf{Dist}$) constraint filtering.

```

1  $length \leftarrow \sum_{\substack{\vec{S} \\ v_i \rightarrow v_j}} d_{i,j}$ 
2 if  $\vec{S}.isFixed()$  then
3    $\mathbf{Dist} \leftarrow length$ 
4 else
5    $\lfloor \mathbf{Dist} \rfloor \leftarrow \max(length, \lfloor \mathbf{Dist} \rfloor)$ 
6    $maxDetour \leftarrow \lceil \mathbf{Dist} \rceil - length$ 
7   for  $v_j \in \vec{S}.getInsertable()$  do
8     for  $v_i \in \vec{S}.getInsert(v_j)$  do
9        $v_k \leftarrow \vec{S}.getNext(v_i)$ 
10       $cost \leftarrow d_{i,j} + d_{j,k} - d_{i,k}$ 
11      if  $cost > maxDetour$  then
12         $\vec{S}.notBetween(v_i, v_j, v_k)$ 

```

Algorithm 5 follows a structure used in most filtering algorithms over sequence variables. The partial sequence $\vec{S}$ is first traversed and potentially used to filter other variables. Then, all insertions are inspected and possibly removed if they cannot be performed.

Regarding constraints in the next sections, the reader is referred to (Delecluse et al. 2022; Thomas et al. 2020) and Appendix C for the presentation of filtering algorithms.

7.2 TransitionTimes

The TransitionTimes constraint is used for problems where node visits involve a service duration and are restricted by time windows, with a transition time required to move from one node to the next. More formally, each node $v_i \in V$ is attached to an integer variable $\mathbf{Time}_i$ representing the start of service at that node and a service duration value s_i . We denote the array of these variables by $\mathbf{Time}$. A matrix $\mathbf{d} \in \mathbb{Z}^{|V| \times |V|}$ defines the transition times between elements and satisfies the triangular inequality. The definition of the constraint is:

$$\text{TransitionTimes}(\vec{S}, \mathbf{Time}, \mathbf{s}, \mathbf{d}) \leftrightarrow \forall v_i \prec_{\vec{S}} v_j : \mathbf{Time}_i + s_i + d_{i,j} \leq \mathbf{Time}_j \quad (36)$$

We consider that waiting at a given node (i.e., reaching it before its time window without beginning the task related to it) is possible, which is why (36) uses inequalities. Moreover, the time variable $\mathbf{Time}_v$ of an unvisited node $v \notin \vec{S}$ is not constrained.

7.3 Precedence

For some applications, visiting a set of nodes in a specific order is important, such as the visit of a pickup that must be done before visiting the corresponding drop-off. The Precedence constraint can be used in such scenarios, ensuring that an ordered set of nodes $\vec{o}$ appears in the same order in a sequence variable ($\vec{o}$ being a fixed sequence, not a variable). It is formally defined as

$$\text{Precedence}(\vec{S}, \vec{o}) \leftrightarrow \forall v_i \prec_{\vec{o}} v_j : v_i, v_j \in \vec{S} \implies v_i \prec_{\vec{S}} v_j \quad (37)$$

Note that some or all nodes in $\vec{o}$ may be absent from the sequence variable. If the nodes from the set $\vec{o}$ must be all present or all absent, one can easily enforce this with $\forall v_i, v_j \in \vec{o} : \mathcal{R}_i(\vec{S}) = \mathcal{R}_j(\vec{S})$.

7.4 Cumulative

Some variations of VRP involve pickups and deliveries, transporting goods or people. The Cumulative constraint can be used to represent those scenarios. It ensures that going through all pickups and deliveries visited in a sequence respects an assigned capacity.

More specifically, let us define an *activity* i as a pair of nodes (p_i, d_i) corresponding to its start (pickup) and end (delivery), respectively. The set of all activities is written A . An activity $i \in A$ consumes a certain load l_i during its execution and can be in one of three states with respect to the partial sequence $\vec{s}$: *fully inserted* if $p_i \in \vec{s} \wedge d_i \in \vec{s}$, *non-inserted* if $p_i \notin \vec{s} \wedge d_i \notin \vec{s}$, and *partially inserted* otherwise (the start or the end is inserted but not both). The Cumulative constraint with a maximum capacity c , with starts $\mathbf{p}$, corresponding ends $\mathbf{d}$ and loads $\mathbf{l}$ is defined as:

$$\text{Cumulative}(\vec{s}, \mathbf{p}, \mathbf{d}, \mathbf{l}, c) \leftrightarrow \begin{cases} \left(\forall v \in \vec{s} : \sum_{i \in A | p_i \preceq v \prec d_i} l_i \leq c \right) \wedge \\ \left(\forall i \in A : p_i \in \vec{s} \iff d_i \in \vec{s} \right) \wedge \\ \left(\forall i \in A : \text{Precedence}(\vec{s}, (p_i, d_i)) \right) \end{cases} \quad (38)$$

This constraint implies precedence constraints between the start p_i and end d_i of an activity $i \in A$ ($\forall i \in A : \text{Precedence}(\vec{s}, (p_i, d_i))$), and that its nodes are either all present or all absent ($\forall i \in A : \mathcal{R}_{p_i}(\vec{s}) = \mathcal{R}_{d_i}(\vec{s})$). An example of a sequence over which the constraint holds is presented in Figure 2.

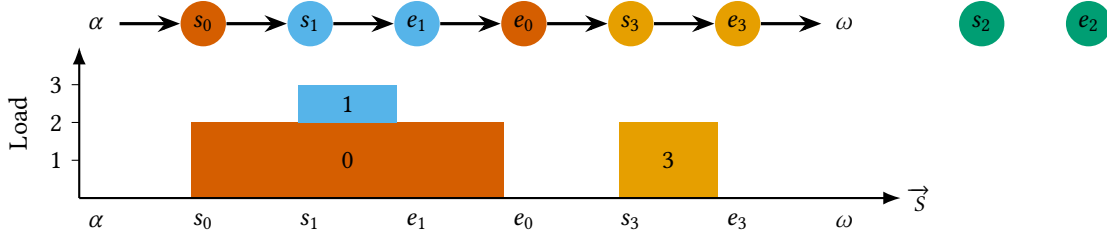


Fig. 2. $\text{Cumulative}(\vec{s}, (s_0, s_1, s_2, s_3), (e_0, e_1, e_2, e_3), (2, 1, 1, 2), 3)$ over a fixed sequence variable $\vec{s} = \alpha \cdot s_0 \cdot s_1 \cdot e_1 \cdot e_0 \cdot s_3 \cdot e_3 \cdot \omega$. The sequence is shown at the top, and the graph below shows the accumulated load for each node $v \in \vec{s}$. Note how nodes s_2, e_2 related to activity 2 are not part of the sequence.

8 Search on a sequence variable

This section presents the basic search procedure used in conjunction with one or more sequence variables to explore the search space.

8.1 Branching

Filtering of the constraints is generally not enough to terminate with fixed sequences. A search procedure is needed to explore the search space. When working with sequence variables, this corresponds to iteratively choosing an unfixed sequence and applying alternative decisions further constraining its domain, through the use of *insert* and *notBetween* operations. Once all sequences in the problem are fixed and the constraints are satisfied, a solution to the problem has been found.

As shown in the left part of Figure 3, different intermediate sequences can ultimately lead to the same one through different insertion steps. This symmetry, induced by branching decisions, can cause inefficiencies. Ideally, we would explore search trees corresponding to disjoint search spaces.

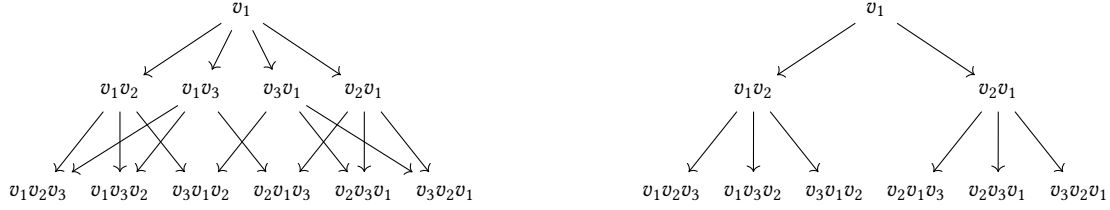


Fig. 3. Sequences created from a fully connected graph $G(V, E)$ with $V = \{\alpha, v_1, v_2, v_3, \omega\}$, where all nodes are required ($V = R$). Nodes α and ω are implicit and not shown. Left: each sequence is extended by inserting any node at each step. Right: a node is first selected for insertion, then all its feasible positions are considered for insertion before moving to the next node (first v_1 , then v_2 , then v_3).

Disjoint search spaces. A simple branching strategy that guarantees the generation of disjoint search spaces is the two-step n -ary branching:

- (1) **Node selection:** Choose an insertable node $v_i \in I$ within a sequence variable $\vec{s}$.
- (2) **Node branching:** For each insertion of v_i in $\vec{s}$, create a corresponding branching decision.

The first step is similar to the variable selection used with integer variables. It allows the integration of first-fail strategies, such as selecting nodes with the fewest possible insertion points. The second step is conceptually similar to value selection; therefore, the most promising insertions should be attempted first. It allows the integration of insertion-based heuristics, such as selecting the insertion that results in the smallest increase in tour length. This strategy was already presented for the DARP in Algorithm 2.

As can be observed in the right part of Figure 3, this strategy generates only distinct sequences. Another simple binary branching strategy that offers the same guarantees is to replace the second step with only two branches. An insertion point is selected, with the insertion performed on the left branch, while the corresponding notBetween constraint is enforced on the right branch.

8.2 Large Neighborhood Search

The usage of LNS with sequence variables was already presented in Algorithm 4. In the context of VRP, this algorithm consists of relaxing some tours by removing nodes from sequences. This approach maintains partial tours, similar in spirit to the partial-order scheduling method introduced for scheduling in (Godard et al. 2005). It also accurately corresponds to the original LNS presented in (Shaw 1998), where partial tours are extended through insertions.

The reconstruction phase uses CP and its search capabilities to reinsert the removed nodes into the restricted problem, possibly within a time limit, before restarting the process.

The set of nodes to relax can be selected in various ways, such as randomly or with more advanced strategies based on relatedness criteria, such as geographical proximity or time-based considerations between nodes, as in (Bent and Van Hentenryck 2004; Christiaens and Vanden Berghe 2020).

9 Experimental Results

We previously presented sequence variables, several global constraints and considerations on the search procedure. We will now use that information to solve routing and sequencing problems.

9.1 Dial-A-Ride Problem

We consider the DARP model from (13)-(22). The state-of-the-art, to the best of our knowledge, is the Adaptive LNS proposed by (Gschwind and Drexel 2019), combining the operators proposed by (Ropke and Pisinger 2006) (random, worst and Shaw removal with greedy and regret based insertions) with additional relaxations, exploiting 9 removal and 5 insertion operators in total. Insertions are evaluated based on a feasibility check specific to the DARP, and obtained solutions close to the best found so far are further optimized using an adaptation of the neighborhood proposed in (Balas and Simonetti 2001). For further reading on the DARP, the reader is referred to the literature review in (Ho et al. 2018).

9.1.1 Search. The search procedure for the DARP experiments is a specific instantiation of the generic sequence branching (Algorithm 2) adapted for pickup and delivery requests, as presented in Algorithm 6. This procedure generates the alternative child nodes of the search tree by considering all valid insertions of both the pickup and delivery nodes of a chosen request into the sequence variables representing the active routes.

Algorithm 6: Creation of the branching points for the DARP in the Sequence Variable Model.

```

1 if  $\bigwedge_{k \in K} \text{Route}_k.\text{isFixed}()$  then
2   return solution
3  $r_i \leftarrow \underset{r_j \in R | r_j \text{ not yet inserted}}{\text{argmin}} \sum_{k \in K} \text{Route}_k.\text{nInsert}(r_j^+) \times \text{Route}_k.\text{nInsert}(r_j^-)$ 
4  $\text{branches} \leftarrow \{\}$ 
5 for  $k \in K$  do
6    $I^+ \leftarrow \text{Route}_k.\text{getInsert}(r_i^+)$ 
7   for  $p^+ \in I^+$  do
8      $I^- \leftarrow \text{Route}_k.\text{getInsertAfter}(r_i^-, p^+) \cup \{r_i^+\}$ 
9     for  $p^- \in I^-$  do
10       $\text{branches} \leftarrow \text{branches} \cup \{(\text{Route}_k.\text{insert}(p^+, r_i^+) \wedge \text{Route}_k.\text{insert}(p^-, r_i^-))\}$ 
11 sort  $\text{branches}$  by increasing heuristic value
12 return  $\text{branches}$ 

```

The DARP branching prioritizes the unvisited request r_i with the fewest remaining feasible insertions (line 3). Given this request, all insertion points I^+ for its pickup r_i^+ are retrieved (line 6), followed by suitable insertions I^- for its delivery r_i^- (line 8). All valid combinations of predecessors are considered as an alternative decision to explore (line 10).

Example 9.1. The example refers to Figure 4 with only one vehicle. A route serves both visits of r_1 . The request r_2 is selected, resulting in six possible sequences for inserting its two visits, each corresponding to a possible branching decision in the search tree. These insertion options are sorted by their cost.

We use a simple LNS that always relaxes 10 requests chosen randomly and uses Algorithm 6 for branching. The request to insert is the one that has the minimum number of insertions, defined as the sum of insertions for its pickup and drop. Insertions with a small increase in distance and high-preserved time slack are considered first, as in (Jain and Van Hentenryck 2011). When considering an insertion of node v_j between nodes v_i, v_k in a vehicle, the heuristic cost is defined as:

$$C_1(d_{i,j} + d_{j,k} - d_{i,k}) - C_2(\lceil \text{Time}_k \rceil - \lfloor \text{Time}_i \rfloor - s_i - d_{i,j} - s_j - d_{j,k}) \quad (39)$$

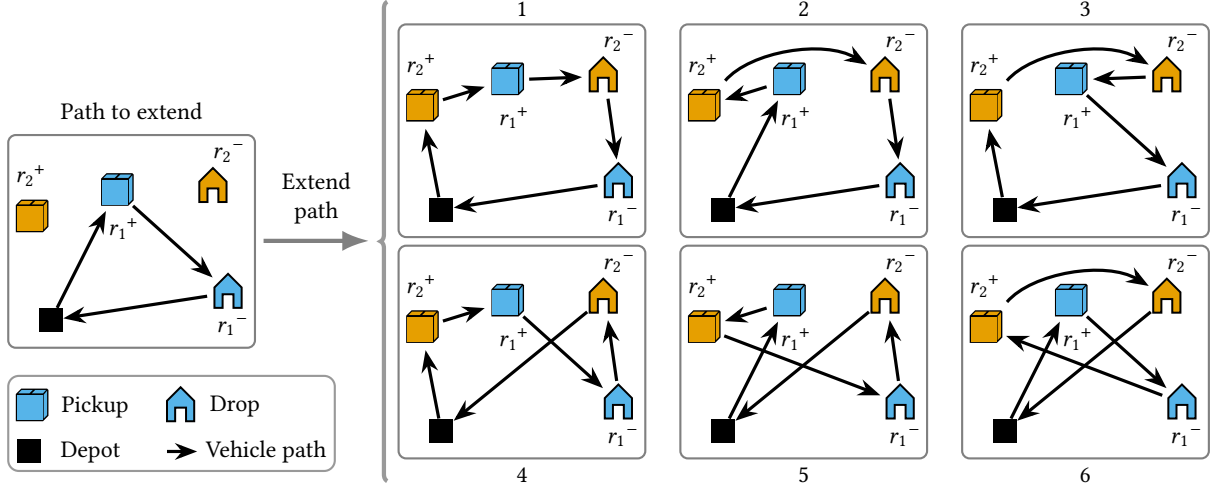


Fig. 4. Paths generated by Algorithm 6 from an initial path (left). Numbers indicate the order in which the paths are considered.

where constants C_1, C_2 control the importance of the detour cost and the preserved time slack, respectively. The heuristic cost for inserting a request $r_i \in R$ is the sum of the costs for inserting its pickup i^+ and drop i^- . We use $C_1 = 80$ and $C_2 = 1$ as in (Jain and Van Hentenryck 2011).

9.1.2 Computational Results. We compare the sequence variable approach with other approaches suitable for modeling VRP:

- **A successor model** written in MiniZinc (Nethercote et al. 2007) (version 2.10.0) and run with the Gecode solver (Schulte, Tack, and Lagerkvist 2010) (version 6.4.0), which performed best on the DARP across all MiniZinc backends. The underlying CP model is essentially the same as in (Berbeglia et al. 2011). Both exhaustive search (**Succ**) and LNS (**Succ-LNS**) are reported.
- **OR-Tools Routing** (Java API, version 9.11.4210), which relies on local search (Cuvelier et al. 2023; Perron and Furnon 2019). All combinations of first solution strategy and local-search metaheuristics in the solver have been tried, and we report the best overall configuration.
- **Hexaly** (version 14.5), a commercial solver specialized in routing (Benoist et al. 2024), using a model provided by the Hexaly team.
- **CP Optimizer (CPO)** (version 22.1.2), a commercial scheduling solver, whose model is written using head-tail sequence variables.
- **The Mixed-Integer Programming model** from (Rist and Forbes 2021), denoted as **RF (2021)**, using the authors' implementation (relying on Python 3.7.0 and Gurobi 9.1.0). This is the state-of-the-art exact method for the DARP. Its decision variables are fragments of paths that are combined, and whose enumeration exploits dominance rules specific to the DARP.

Details on the models are in Appendix D. All experiments were conducted on a machine running Ubuntu 24.04, equipped with an AMD Ryzen™ 9 9950X processor (16 cores, 32 threads, 1280 KB of L1 cache, 16 MB of L2 cache, and 64 MB of L3 cache) and 64 GB of RAM. No GPU was used. The Java-based implementations were

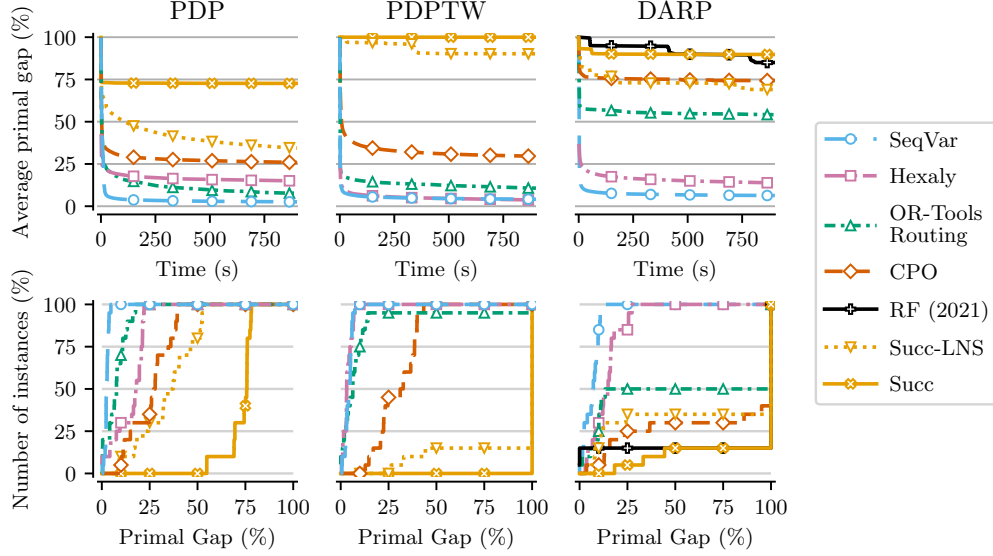


Fig. 5. Primal-gap performance on the PDP, PDPTW, and DARP. The top row shows the mean primal gap over time; lower-left curves indicate faster convergence. The bottom row shows the percentage of instances with final gap at most the x-axis value after 15 minutes; upper-left curves are better.

executed with OpenJDK 25.0.3; the sequence variable was implemented in Java using MaxiCP (Schaus et al. 2024), an extended and open-source version of MiniCP (Michel et al. 2021).

GNU Parallel (version 20231122) (Tange 2023) was used to execute 32 single-threaded solver processes concurrently. A time limit of 15 minutes was imposed for each run. Each solver was run 10 times per instance, and all reported averages are computed over these 10 runs.

The results for the 20 instances from (Cordeau and Laporte 2003) are shown in Figure 5. The top row shows the average primal gap to the best known solution over time for each solver, where the gap is first averaged over runs for each instance and then averaged over all instances. The bottom row shows the proportion of instances whose final average gap is at most τ : $\Gamma_s(\tau) = |\{p \in \mathcal{P} : \gamma_{p,s} \leq \tau\}|/|\mathcal{P}|$, where $\mathcal{P}$ denotes the set of problem instances and $\gamma_{p,s}$ denotes the final primal gap of solver s on instance p , averaged over all runs. In both plots, the gap is set to 1 for any run in which no incumbent solution is found.

The approach using a sequence variable, although based on a simple LNS, consistently achieves solutions within 15% of the best-known results for the DARP. The exact approach RF (2021) is the best performer on 3 instances only, and fails to provide any feasible solution on the remaining instances, as the enumeration of the fragments of paths times out. Detailed results per instance are provided in Appendix D.1.

We also report performance when (i) the ride time constraints and maximum route duration of the DARP are removed—reducing it to a Pickup and Delivery Problem with Time Windows (PDPTW)—and (ii) when time windows are additionally removed—reducing it further to a Pickup and Delivery Problem (PDP). The RF (2021) method, however, is only reported on the DARP, as it requires substantial changes to be adapted to the PDP and PDPTW. These VRP variants illustrate the adaptability of the proposed models. For these cases as well, the best results are obtained using the sequence variable approach.

9.2 A Prize Collecting Scheduling Problem

Although this problem is not a VRP, it illustrates the applicability of sequence variables to scheduling problems.

The Prize-Collecting Job Sequencing with One Common and Multiple Secondary Resources (PC-JSOCMSR) is a scheduling problem in which the objective is to maximize the reward collected from the selected tasks, while satisfying several resource and timing constraints. It was introduced in (Horn, Raidl, et al. 2018) and is motivated by the scheduling of cancer patients requiring particle therapy. The problem is defined on a set of n tasks $t_i \in T$, each with processing duration d_i and reward z_i when executed. Every task requires the global resource and one particular secondary resource $j \in R$, where R is the set of secondary resources. The tasks associated with secondary resource j are denoted by $T(j)$. Neither the global resource nor any secondary resource can process more than one task simultaneously. Moreover, executing a task t_i reserves its secondary resource for a pre-processing duration d_i^{pre} and a post-processing duration d_i^{post} . Each task must be executed non-preemptively in one of its ω_i time windows $w_i = \{w_{ik} \mid k = 0, \dots, \omega_i - 1\}$, where $w_{ik} = [w_{ik}^{start}, w_{ik}^{end}]$. The objective is to maximize the total reward of the executed tasks.

To simplify the naming, the problem will be abbreviated to the Prize Collecting Scheduling Problem (PCSP) instead of PC-JSOCMSR. An example of a PCSP solution is presented in Figure 6.

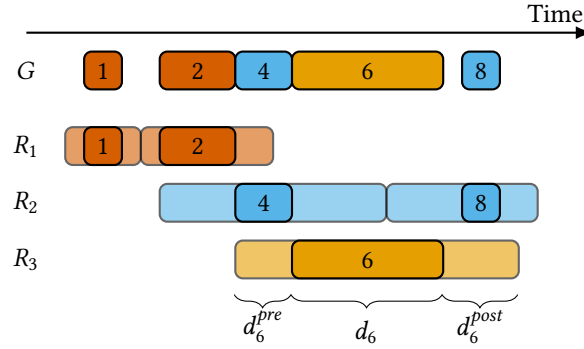


Fig. 6. PCSP solution

The best published approaches for this problem rely on Mixed Integer Linear Programming for exact search, and local search for heuristic approaches for the largest instances (Froger and Sadykov 2023). Some other methods, using dynamic programming, decision diagrams, or CP, have also been developed in (Horn, Maschler, et al. 2021).

Model. Our model for the PCSP is as follows:

$$\max \sum_{t_i \in T} z_i \cdot \mathcal{R}_{t_i}(\mathbf{G}) \quad (40)$$

subject to:

$$\text{TransitionTimes}(\mathbf{G}, \mathbf{Time}, \mathbf{d}, \mathbf{O}) \quad (41)$$

$$\text{TransitionTimes}(\mathbf{R}_j, \mathbf{Time}, \mathbf{d}, \mathbf{Tr}) \quad \forall j \in R \quad (42)$$

$$\text{SubSequence}(\mathbf{G}, \mathbf{R}_j) \quad \forall j \in R \quad (43)$$

$$\mathcal{R}_{t_i}(\mathbf{R}_j) = \mathcal{R}_{t_i}(\mathbf{G}) \quad \forall j \in R, \forall t_i \in T(j) \quad (44)$$

$$\mathcal{R}_{t_i}(\mathbf{R}_j) = 0 \quad \forall j \in R, \forall t_i \in (T \setminus T(j)) \quad (45)$$

$$\mathbf{Time}_i \in w_i \quad \forall t_i \in T \quad (46)$$

$$\text{first}(\mathbf{G}) = \alpha, \quad \text{last}(\mathbf{G}) = \omega \quad (47)$$

$$\text{first}(\mathbf{R}_j) = \alpha, \quad \text{last}(\mathbf{R}_j) = \omega \quad \forall j \in R \quad (48)$$

The model uses one sequence variable for each resource: $\mathbf{G}$ represents the global resource, while $\mathbf{R}_j$ represents the secondary resource $j \in R$. The endpoints of these sequences are dummy nodes. The objective maximizes the total reward of the executed tasks (40). TransitionTimes constraints enforce the time-window requirements on both the global sequence (41) and the secondary sequences (42). For each task t_i , $\mathbf{Time}_i$ is derived from its original time windows w_i (46); together, these variables form the array $\mathbf{Time}$. The transition times on the global resource are all zero and are represented by the matrix $\mathbf{O} \in \mathbb{Z}^{T \times T}$ (41). For a secondary resource, the transition-time matrix $\mathbf{Tr} \in \mathbb{Z}^{T \times T}$ has entries $\text{Tr}_{i,i'} = d_i^{\text{post}} + d_{i'}^{\text{pre}}$. Consistency between the global and secondary resource schedules is imposed by the SubSequence constraint (43), which requires $\vec{\mathbf{R}}_j$ to be a subsequence of $\mathbf{G}$: $\mathbf{R}_j \subseteq \mathbf{G}$. It can be viewed as a special case of the Precedence constraint from Section 7.3, in which the subsequence is itself a sequence variable. Further details on this constraint are provided in Appendix C.4. If a task is executed on the global resource, it must also be executed on its associated secondary resource (44). Finally, a task cannot be executed on a secondary resource with which it is not associated (45).

Search. We use the same LNS framework as for the DARP, but considering each task individually instead of working with requests. The first incumbent is obtained with a simple constructive search that explicitly handles optional tasks: required-but-uninserted tasks are inserted first, selecting the one with the fewest insertion points. Otherwise, the optional task with the largest reward is selected. The branching enumerates all feasible insertion positions and, for optional tasks, one additional exclusion branch. This is similar to the enumeration of branching choices from Algorithm 2.

During the LNS repair search, we use a binary insertion branching. The task to insert is selected according to the number of available insertions scaled by its reward. Its insertion position is chosen with the same distance/slack criterion as in Equation (39), using transition times on the considered sequence. The two branches either insert the task at its best position or forbid this insertion.

9.2.1 Computational Results. We use the instances from (Froger and Sadykov 2023) and report the quality of the found solutions relative to the best solutions reported by the authors. Given the large dataset available (1200 instances), we restricted ourselves to 3 instances per number of tasks and secondary resources, for $n \in \{50, 100, 150, 200, 250, 300, 400, 500\}$ and $m \in \{3, 4\}$ (Avionic dataset) or $m \in \{2, 3\}$ (Therapy dataset). This leaves 96 instances ranging from 50 to 500 tasks and 2 to 4 secondary resources.

We compare to the same technologies as in Section 9.1. For MiniZinc, we compare to both a successor model (MZN-Succ) and the model from (Horn, Raidl, et al. 2018) (MZN-Disj), which model each machine’s occupancy using disjunctive constraints and prevent the use of a transition-time matrix. We used the Chuffed solver (Chu et al. 2018) in both cases, which outperformed the other ones for the successor model and was already used in (Horn, Raidl, et al. 2018). LNS is not shown for MiniZinc as the Chuffed backend does not implement it.

Figure 7 shows the average gap over time (left) and the final distribution (right) over the instances. The CPO model is the clear winner, while the sequence variable is a close second. Compared to the DARP, the routing models from Hexaly and OR-Tools lag further behind the sequence variable approach, and OR-Tools dominates Hexaly. As for the DARP, the MiniZinc models are the least performant out of the tried approaches. A detailed breakdown of the final gap values per instance size is provided in Appendix E, along with further details on the set of instances used.

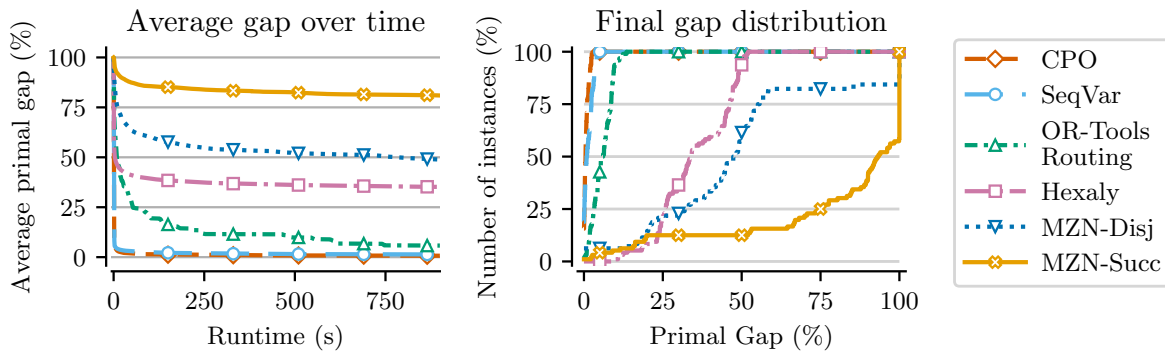


Fig. 7. Primal-gap performance on the PCSP. The left plot shows the mean primal gap over time; lower-left curves indicate faster convergence. The right plot shows the percentage of instances with final gap at most the x-axis value after 15 minutes; upper-left curves are better.

10 Conclusion

This paper enhances previous proposals of sequence variables, which are used in CP to tackle vehicle routing and sequencing problems. Their domain representation, implementation, and interactions with Boolean variables are proposed and formalized. They ease the modeling of complex VRPs such as the DARP while staying close to the state of the art in terms of performance. These variables are compatible with optional visits, insertion-based heuristics, and can be easily combined with Large Neighborhood Search, making them practical for solving complex VRPs in a CP framework.

Acknowledgments

This work was supported by Service Public de Wallonie Recherche under grant n°2010235 – ARIAC by DIGITAL-WALLONIA4.A

The authors would like to thank the Hexaly team for providing their model of the DARP, as well as a license key for running their solver.

References

- E. Balas and N. Simonetti. 2001. “Linear time dynamic-programming algorithms for new classes of restricted TSPs: A computational study.” *INFORMS journal on Computing*, 13, 1, 56–75.

- P. Benchimol, W.-J. v. Hoeve, J.-C. Régin, L.-M. Rousseau, and M. Rueher. 2012. "Improved filtering for weighted circuit constraints." *Constraints*, 17, 205–233.
- T. Benoist, F. Gardi, D. Julien, and R. Megel. Oct. 2024. *Hexaly*. (Oct. 2024). <https://www.hexaly.com>.
- R. Bent and P. Van Hentenryck. 2004. "A Two-Stage Hybrid Local Search for the Vehicle Routing Problem with Time Windows." *Transportation Science*, 38, 4, 515–530.
- G. Berbeglia, G. Pesant, and L.-M. Rousseau. 2011. "Checking the feasibility of dial-a-ride instances using constraint programming." *Transportation Science*, 45, 3, 399–412.
- F. Boussemart, F. Hemery, C. Lecoutre, and L. Sais. 2004. "Boosting systematic search by weighting constraints." In: *ECAI* 146. Vol. 16, 96–97.
- F. Boussemart, C. Lecoutre, G. Audemard, and C. Piette. 2016. "XCSP3: an integrated format for benchmarking combinatorial constrained problems." *arXiv preprint arXiv:1611.03398*.
- K. Braekers, K. Ramaekers, and I. Van Nieuwenhuysse. 2016. "The vehicle routing problem: State of the art classification and review." *Computers & industrial engineering*, 99, 300–313.
- Q. Cappare, C. Thomas, P. Schaus, and L.-M. Rousseau. 2018. "A Constraint Programming Approach for Solving Patient Transportation Problems." In: *International Conference on Principles and Practice of Constraint Programming (CP)*. Ed. by J. Hooker. Springer, 490–506.
- J. Christiaens and G. Vanden Berghe. 2020. "Slack induction by string removals for vehicle routing problems." *Transportation Science*, 54, 2, 417–433.
- G. Chu, P. J. Stuckey, A. Schutt, T. Ehlers, G. Gange, and K. Francis. 2018. *Chuffed: A Lazy Clause Generation Solver*. <https://github.com/chuffed/chuffed>. GitHub repository. (2018).
- J.-F. Cordeau and G. Laporte. July 2003. "A tabu search heuristic for the static multi-vehicle dial-a-ride problem." *Transportation Research Part B: Methodological*, 37, (July 2003), 579–594. doi:10.1016/S0191-2615(02)00045-0.
- T. Cuvelier, F. Didier, V. Furnon, S. Gay, S. Mohajeri, and L. Perron. 2023. "Or-tools' vehicle routing solver: A generic constraint-programming solver with heuristic search for routing problems." In: *24e congrès annuel de la société française de recherche opérationnelle et d'aide à la décision*.
- A. Delecluse, P. Schaus, and P. Van Hentenryck. 2022. "Sequence Variables for Routing Problems." In: *International Conference on Principles and Practice of Constraint Programming (CP)*.
- G. Dooms, Y. Deville, and P. Dupont. 2005. "CP(Graph): Introducing a Graph Computation Domain in Constraint Programming." In: *International Conference on Principles and Practice of Constraint Programming (CP)*. Springer, 211–225.
- A. Froger and R. Sadykov. 2023. "New exact and heuristic algorithms to solve the prize-collecting job sequencing problem with one common and multiple secondary resources." *European Journal of Operational Research*, 306, 1, 65–82.
- C. Gervet. 1997. "Interval propagation to reason about sets: Definition and implementation of a practical language." *Constraints*, 1, 191–244.
- D. Godard, P. Laborie, and W. Nuijten. 2005. "Randomized Large Neighborhood Search for Cumulative Scheduling." In: *International Conference on Automated Planning and Scheduling (ICAPS)*. Vol. 5. AAAI-Press, 81–89.
- T. Gschwind and M. Drexl. 2019. "Adaptive large neighborhood search with a constant-time feasibility test for the dial-a-ride problem." *Transportation Science*, 53, 2, 480–491.
- R. Hartert, P. Schaus, S. Vissicchio, and O. Bonaventure. 2015. "Solving segment routing problems with hybrid constraint programming techniques." In: *International Conference on Principles and Practice of Constraint Programming (CP)*. Springer, 592–608.
- P. V. Hentenryck. 2002. "Constraint and integer programming in OPL." *INFORMS Journal on Computing*, 14, 4, 345–372.
- S. C. Ho, W. Y. Szeto, Y.-H. Kuo, J. M. Leung, M. Petering, and T. W. Tou. 2018. "A survey of dial-a-ride problems: Literature review and recent developments." *Transportation Research Part B: Methodological*, 111, 395–421.
- M. Horn, J. Maschler, G. R. Raidl, and E. Rönnberg. 2021. "A*-based construction of decision diagrams for a prize-collecting scheduling problem." *Computers & Operations Research*, 126, 105125.
- M. Horn, G. Raidl, and C. Blum. 2018. "Job sequencing with one common and multiple secondary resources: A problem motivated from particle therapy for cancer treatment." In: *Machine Learning, Optimization, and Big Data: Third International Conference, MOD 2017, Volterra, Italy, September 14–17, 2017, Revised Selected Papers 3*. Springer, 506–518.
- S. Jain and P. Van Hentenryck. 2011. "Large Neighborhood Search For Dial-a-Ride Problems." In: *International Conference on Principles and Practice of Constraint Programming (CP)*. Springer, 400–413.
- P. Kilby, P. Prosser, and P. Shaw. 2000. "A comparison of traditional and constraint-based heuristic methods on vehicle routing problems with side constraints." *Constraints*, 5, 4, 389–414.
- P. Kilby and P. Shaw. 2006. "Vehicle routing." In: *Handbook of Constraint Programming*. Elsevier, 801–836.
- P. Laborie, J. Rogerie, P. Shaw, and P. Vilím. 2009. "Reasoning with Conditional Time-Intervals. Part II: An Algebraical Model for Resources." In: *FLAIRS Conference*.
- P. Laborie, J. Rogerie, P. Shaw, and P. Vilím. Apr. 2018. "IBM ILOG CP Optimizer for Scheduling." *Constraints*, 23, 2, (Apr. 2018), 210–250. doi:10.1007/s10601-018-9281-x.
- J. Lauriere. 1978. "A Language and a Program for Stating and Solving Combinatorial Problems." *Artificial Intelligence*, 10, 1, 29–127. doi:10.1016/0004-3702(78)90029-2.

- C. Liu, D. M. Aleman, and J. C. Beck. 2018. “Modelling and solving the senior transportation problem.” In: *International Conference on the Integration of Constraint Programming, Artificial Intelligence, and Operations Research (CPAIOR)*. Springer, 412–428.
- L. Michel, P. Schaus, and P. Van Hentenryck. 2021. “MiniCP: a lightweight solver for constraint programming.” *Mathematical Programming Computation*, 13, 1, 133–184. ISBN: 1867-2957. doi:10.1007/s12532-020-00190-7.
- N. Nethercote, P. J. Stuckey, R. Becket, S. Brand, G. J. Duck, and G. Tack. 2007. “MiniZinc: Towards a standard CP modelling language.” In: *International Conference on Principles and Practice of Constraint Programming (CP)*. Springer, 529–543.
- L. Perron and V. Furnon. 2019. *OR-Tools*. Version 7.2. Google, (2019). <https://developers.google.com/optimization/>.
- G. Pesant, M. Gendreau, and J.-M. Rousseau. 1997. “GENIUS-CP: A generic single-vehicle routing algorithm.” In: *International Conference on Principles and Practice of Constraint Programming (CP)*. Springer, 420–434.
- J.-F. Puget. 1993. “Set constraints and cardinality operator: Application to symmetrical combinatorial problems.” In: *Third Workshop on Constraint Logic Programming–WCLP93*, 211.
- Y. Rist and M. A. Forbes. 2021. “A new formulation for the dial-a-ride problem.” *Transportation Science*, 55, 5, 1113–1135.
- S. Ropke and D. Pisinger. 2006. “An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows.” *Transportation science*, 40, 4, 455–472.
- D. J. Rosenkrantz, R. E. Stearns, and P. M. Lewis II. 1977. “An analysis of several heuristics for the traveling salesman problem.” *SIAM Journal on Computing*, 6, 3, 563–581.
- V. I. C. de Saint-Marçq, P. Schaus, C. Solnon, and C. Lecoutre. 2013. “Sparse-Sets for Domain Implementation.” In: *CP workshop on Techniques for Implementing Constraint programming Systems (TRICS)*, 1–10.
- M. W. Savelsbergh. 1985. “Local search in routing problems with time windows.” *Annals of Operations Research*, 4, 1, 285–305.
- P. Schaus, G. Derval, A. Delecluse, L. Michel, and P. V. Hentenryck. 2024. *MaxiCP: A Constraint Programming Solver for Scheduling and Vehicle Routing*. (2024). <https://github.com/aia-uclouvain/maxicp>.
- C. Schulte and G. Tack. 2013. “View-based propagator derivation.” *Constraints*, 18, 75–107.
- C. Schulte, G. Tack, and M. Z. Lagerkvist. 2010. “Modeling and Programming with Gecode.”
- P. Shaw. 1998. “Using constraint programming and local search methods to solve vehicle routing problems.” In: *International Conference on Principles and Practice of Constraint Programming (CP)*. Springer, 417–431.
- O. Tange. Nov. 2023. *GNU Parallel 20231122 (“Grindavik”)*. (Nov. 2023). doi:10.5281/zenodo.10199085.
- C. Thomas, R. Kameugne, and P. Schaus. 2020. “Insertion Sequence Variables for Hybrid Routing and Scheduling Problems.” In: *International Conference on Integration of Constraint Programming, Artificial Intelligence, and Operations Research (CPAIOR)*. Springer, 457–474.
- P. Van Hentenryck and L. Michel. 2014. “Domain views for constraint programming.” In: *International Conference on Principles and Practice of Constraint Programming (CP)*. Springer, 705–720.

A Domain Implementation Details

A.1 Initialization

When initialized, a compact sequence domain is defined on the set of nodes V belonging to the graph $G(V, E)$, and its successor array s^+ contains one entry per node $v \in V$. The two starting and ending nodes, α and ω , are identified upon the initialization of the domain. These nodes constitute the initial sequence of nodes represented by the domain, which form a cycle in the successor array.

$$s_\alpha^+ = \omega \wedge s_\omega^- = \alpha \wedge s_\omega^+ = \alpha \wedge s_\alpha^- = \omega \quad (49)$$

Note that a link from the end node ω to the first node α is encoded, thus representing a cycle instead of a path, to ease invariant encoding in further equations. The other nodes point to themselves as self-loops. At initialization, the edge set forms a complete graph, with the exception of edges that originate at ω and end at α : $E = \{(v_1, v_2) \mid v_1, v_2 \in V : v_1 \neq \omega \wedge v_2 \neq \alpha \wedge v_1 \neq v_2\} \cup \{(\omega, \alpha)\}$. The set of nodes that may be inserted is defined as $I = V \setminus \{\alpha, \omega\}$.

A.2 Invariants

First, we introduce one predicate that will be instrumental in defining the consistency invariants. It determines (in constant time) if a node v_i belongs to the partial sequence (i.e., if $v_i \in \vec{s}$ holds) by verifying if it is not a self-loop:

$$\text{isMember}(v_i) \equiv s_i^+ \neq v_i. \quad (50)$$

The lower-level consistency invariants expressed on the data structures are given next. We first identify the invariants describing the channeling between some data structures.

$$\forall v_i, v_j \in V : v_i \in E_j^- \iff v_j \in E_i^+ \quad (51)$$

$$\forall v_i, v_j \in V : s_i^+ = v_j \iff s_j^- = v_i \quad (52)$$

The invariant (51) ensures that each edge (v_i, v_j) appears twice in the data structures: one for the adjacency set of the incoming node, and one for the outgoing node. The invariant (52) ensures that for any two nodes v_i and v_j in the graph, v_i is the predecessor of v_j if and only if v_j is the successor of v_i . It guarantees a consistent and mutual relationship between successor and predecessor links for every node in the graph. In addition to channeling, specific invariants are used to maintain the counters previously introduced.

$$nS = |\{v_i \mid v_i \in V \wedge \text{isMember}(v_i)\}| \quad (53)$$

$$\forall v_j \in I : nI_j = |\{v_i \mid v_i \in E_j^- \wedge \text{isMember}(v_i)\}| \quad (54)$$

$$\forall v_i \in V : (nI_i \geq 1) \leftrightarrow (v_i \in I) \quad (55)$$

The length of the partial sequence is tracked in (53). Invariant (54) tracks how many insertions are feasible for a given node $v_j \in I$. This counter is used to ensure that every node $v_j \in I$ has at least one possible insertion (55): otherwise, the node is not insertable and thus has zero insertions.

Given the array of successors s^+ , one can define the set of nodes reachable in the circuit containing node $v_i \in V$:

$$\text{circuit}(v_i) = \text{circuit}(v_i, \emptyset) \quad (56)$$

$$\text{circuit}(v_i, S) = \begin{cases} S & \text{if } v_i \in S \\ \text{circuit}(s_i^+, S \cup \{v\}) & \text{otherwise} \end{cases} \quad (57)$$

Intuitively, from (56), $\text{circuit}(v_i)$ returns all nodes in the sequence from α to ω if v_i is in the sequence; otherwise, it returns a set containing only v_i . This is done by recursively following the pointers s^+ of the successor array. Using this definition, the implementation invariants are as follows.

$$s_\omega^+ = \alpha \wedge s_\alpha^- = \omega \quad (58)$$

$$\forall v_i, v_j \in V, v_i \neq v_j : (s_i^+ = v_j \implies v_j \in E_i^+) \wedge (s_j^- = v_i \implies v_i \in E_j^-) \quad (59)$$

$$\forall v_i, v_j \in V, v_i \neq v_j : s_i^+ \neq s_j^+ \quad (60)$$

$$\forall v_i \in V : \text{isMember}(v_i) \iff \text{circuit}(v_i) = \text{circuit}(\alpha) \quad (61)$$

$$\forall v_i, v_j \in V, v_i \neq v_j : \neg \text{isMember}(v_i) \wedge \neg \text{isMember}(v_j) \implies v_i \in E_j^- \quad (62)$$

$$\forall v_i, v_j, v_k \in V, v_i \neq v_j \neq v_k : s_i^+ = v_k \implies (v_i \in E_j^- \iff v_k \in E_j^+) \quad (63)$$

Invariant (58) ensures that the successor of the last node ω being visited always points toward the first node α . Invariant (59) enforces that the current successor of a node $v_i \in \vec{s}$ exists within the outgoing edges of the node v_i . Invariants (60) and (61) ensure that only one sub-circuit is encoded within the successor array. All successors must be different, and if the successor of a node $v_i \in V$ is set (i.e. v_i belongs to the partial sequence) then v_i belongs to the circuit of the first node α (and the last node ω given that $\text{circuit}(\alpha) = \text{circuit}(\omega)$). Invariant (62) enforces that all insertable nodes form a clique. Finally, (63) ensures that the two edges $(v_i, v_j), (v_j, v_k)$ needed to insert a node $v_j \in I$ after node v_i (with v_k being the current successor of v_i) are either both absent or both present.

Lastly, some invariants interact specifically with the required R and excluded nodes X . Moreover, the implementation may itself modify the set of required nodes R , so that it always contains nodes from the partial

sequence: $\vec{s} \subseteq R$, given that nodes that are part of the partial sequence are always visited in all sequences in the domain. The invariants are:

$$v_i \in X \iff E_i^- = \emptyset \iff E_i^+ = \emptyset \quad (64)$$

$$\forall v_i \in V : \text{isMember}(v_i) \implies v_i \in R \quad (65)$$

$$\forall v_i \in R \cap I : nI_i > 1 \quad (66)$$

Invariant (64) ensures that an excluded node has no edge attached to it. Invariant (65) captures the fact that nodes that are part of the partial sequence $\vec{s}$ are always visited, and thus considered as mandatory. Finally, invariant (66) guarantees that required nodes not part of the sequence have at least two insertions remaining. Otherwise, if only one insertion remained for such a node, it would directly be used to add the node to the partial sequence $\vec{s}$.

B Domain updates

B.1 Insertion

Algorithm 7 is used to perform an $Sq.\text{insert}(v_1, v_2)$ operation on a compact sequence domain Sq , provided that (v_1, v_2) is a feasible insertion. The inserted node v_2 is first marked as required, removed from the insertable nodes and the size nS of the partial sequence $\vec{s}$ is increased (lines 1 to 4). Then, every node v_i linked to v_2 is inspected (line 6). If a node v_i belongs to both the partial sequence $\vec{s}$ and to the incoming edges of v_2 , it could previously be used to perform an insertion using (v_i, v_2) . Given that v_2 is already inserted, such edges are removed (lines 8 to 11). Otherwise, if v_i does not belong to $\vec{s}$, two situations may occur: either v_i cannot be placed directly after v_1 , and thus cannot be placed directly after v_2 either, according to (32) (lines 13 and 14); or the node v_i can be inserted directly after v_1 and can now be inserted directly after v_2 as well, increasing its counter nI_i (line 16). Lastly, the edges, successor and predecessor of v_1, v_2 and v_3 (the previous successor of v_1) are updated to reflect the insertion (lines 17 to 19).

In the implementation, if (v_1, v_2) does not define a feasible insertion, two situations may occur:

- (1) If v_2 is already within the partial sequence and lies after v_1 (i.e. $v_1, v_2 \in \vec{s} \wedge v_1 \prec v_2$), the insertion is considered as having already been performed. Nothing happens in this case.
- (2) Otherwise, v_2 cannot be inserted. This corresponds to a domain wipeout.

B.2 NotBetween

The algorithm 8 performs a $Sq.\text{notBetween}(v_1, v_2, v_3)$ operation on a compact sequence domain Sq . It iterates over the nodes v_i between the nodes v_1 and v_3 (line 1), removes the edges allowing the insertion of v_2 after v_i , and decrements its insertion counter nI_2 (lines 4 to 6). If the insertion counter reaches 0, the node must be excluded due to (55), therefore removing the node v_2 from the set of insertable nodes I and marking it as excluded (lines 8 and 9). In this case, all edges passing through the node are removed. In contrast, if the insertion counter reaches 1 and the node is required (line 14), it is automatically inserted at its only remaining insertion point (lines 15 and 16).

The algorithm 8 is frequently called with v_3 being the direct successor of v_1 : $v_1 \rightarrow v_3$. If so, only one iteration occurs at line 1, with $v_i = v_1$, and edge deletion occurs in constant time if v_2 is not excluded or inserted.

A specific situation must be checked if v_2 already belongs to the partial sequence. If the nodes are consecutive in the partial sequence ($v_1, v_2, v_3 \in \vec{s} \wedge v_1 \prec v_2 \prec v_3$), a domain wipeout is triggered. No filtering occurs if $v_3 \preceq v_1$.

Algorithm 7: $Sq.insert(v_1, v_2)$ **Input:** Sq : compact sequence domain, (v_1, v_2) feasible insertion to perform

```

1  $R \leftarrow R \cup \{v_2\}$ 
2  $I \leftarrow I \setminus \{v_2\}$ 
3  $nI_2 \leftarrow 0$ 
4  $nS \leftarrow nS + 1$ 
5  $v_3 \leftarrow s_1^+$ 
6 for  $v_i \in E_2^-$  do
7   if  $Sq.isMember(v_i)$  then
8     if  $v_i \neq v_1$  then
9        $E_i^+ \leftarrow E_i^+ \setminus \{v_2\}, E_2^- \leftarrow E_2^- \setminus \{v_i\}$ 
10       $v_j \leftarrow s_i^+$ 
11       $E_2^+ \leftarrow E_2^+ \setminus \{v_j\}, E_j^- \leftarrow E_j^- \setminus \{v_2\}$ 
12    else if not  $Sq.canInsert(v_1, v_i)$  then
13       $E_2^+ \leftarrow E_2^+ \setminus \{v_i\}, E_i^- \leftarrow E_i^- \setminus \{v_2\}$ 
14       $E_i^+ \leftarrow E_i^+ \setminus \{v_2\}, E_2^- \leftarrow E_2^- \setminus \{v_i\}$ 
15    else
16       $nI_i \leftarrow nI_i + 1$ 
17  $s_1^+ \leftarrow v_2, s_2^+ \leftarrow v_3$ 
18  $s_3^- \leftarrow v_2, s_2^- \leftarrow v_1$ 
19  $E_1^+ \leftarrow E_1^+ \setminus \{v_3\}, E_3^- \leftarrow E_3^- \setminus \{v_1\}$ 

```

B.3 Require

Requiring a node v_i is obtained by marking it as required ($R \leftarrow R \cup \{v_i\}$) and inserting it if only one insertion remains for it ($nI_i = 1$), similarly to lines 15 and 16 of Algorithm 8. If the node is excluded, a domain wipeout occurs.

B.4 Exclude

The exclusion of a node v_i is obtained by marking it as excluded ($X \leftarrow X \cup \{v_i\}$) and removing all edges connected to it. Although edge removal occurs, only the insertion counter nI_i changes. The counters related to other nodes $v_j \in I \wedge v_i \neq v_j$ are not affected by edge removal, since any edge (v_i, v_j) (or (v_j, v_i)) being removed could not define an insertion: no endpoint of the edge is within the partial sequence $\vec{s}$. If the node is required, a domain wipeout occurs.

C Filtering implementations**C.1 TransitionTimes**

The filtering occurs in two steps. First, the bounds of the time windows of visited nodes are updated by iterating over the partial sequence $\vec{s}$:

$$\lfloor \mathbf{Time}_j \rfloor \leftarrow \max(\lfloor \mathbf{Time}_j \rfloor, \lfloor \mathbf{Time}_i \rfloor + s_i + d_{i,j}) \quad \forall v_i \rightarrow v_j \quad (67)$$

$$\lceil \mathbf{Time}_i \rceil \leftarrow \min(\lceil \mathbf{Time}_i \rceil, \lceil \mathbf{Time}_j \rceil - s_i - d_{i,j}) \quad \forall v_i \rightarrow v_j \quad (68)$$

Algorithm 8: $Sq.\text{notBetween}(v_1, v_2, v_3)$

Input: Sq : compact sequence domain, v_1 and v_3 are two nodes in the partial sequence $\vec{s}$ between which node $v_2 \in V \setminus \vec{s}$ cannot appear.

```

1 for  $v_i \in \vec{s} \mid v_1 \preceq v_i \prec v_3$  do
2   if  $Sq.\text{canInsert}(v_i, v_2)$  then
3      $v_j \leftarrow s_i^+$ 
4      $E_2^- \leftarrow E_2^- \setminus \{v_i\}, E_i^+ \leftarrow E_i^+ \setminus \{v_2\}$ 
5      $E_j^- \leftarrow E_j^- \setminus \{v_2\}, E_2^+ \leftarrow E_2^+ \setminus \{v_j\}$ 
6      $nI_2 \leftarrow nI_2 - 1$ 
7     if  $nI_2 = 0$  then
8        $X \leftarrow X \cup \{v_2\}$ 
9        $I \leftarrow I \setminus \{v_2\}$ 
10      for  $v_k \in I$  do
11         $E_k^- \leftarrow E_k^- \setminus \{v_2\}, E_k^+ \leftarrow E_k^+ \setminus \{v_2\}$ 
12         $E_2^- \leftarrow \emptyset, E_2^+ \leftarrow \emptyset$ 
13      return
14 if  $nI_2 = 1$  and  $v_2 \in R$  then
15    $\{v_i\} \leftarrow Sq.\text{getInsert}(v_2)$ 
16    $Sq.\text{insert}(v_i, v_2)$ 

```

This step can be implemented in $O(|\vec{s}|)$. Next, all insertions of a node $v_j \in I$ between consecutive nodes $v_i, v_k \in \vec{s}, v_i \rightarrow v_k$ are inspected, similarly to line 8 of Algorithm 5. Each insertion can be used to define ea and la : the earliest and latest arrival at node v_j , if performed.

$$ea = \lfloor \text{Time}_i \rfloor + s_i + d_{i,j} \quad (69)$$

$$la = \lceil \text{Time}_k \rceil - s_j - d_{j,k} \quad (70)$$

Insertions corresponding to time-window violations are removed:

$$(ea > \lceil \text{Time}_j \rceil) \vee (la < \lfloor \text{Time}_j \rfloor) \vee (ea > la) \implies \vec{s}.\text{notBetween}(v_i, v_j, v_k) \quad (71)$$

In (71), if either reaching node v_j cannot be done within its time window, or if doing a detour through it would violate the time window of v_k , the insertion is removed. This shares some similarities with the work from (Savelsbergh 1985), where time windows are used for checking the validity of moves in local search. Finally, a time window update is performed for nodes $v_j \in R \setminus \vec{s}$ that are required but not yet inserted:

$$\lfloor \text{Time}_j \rfloor \leftarrow \max \left(\lfloor \text{Time}_j \rfloor, \min_{v_i \in \vec{s}.\text{getInsert}(v_j)} \lfloor \text{Time}_i \rfloor + s_i + d_{i,j} \right) \quad (72)$$

$$\lceil \text{Time}_j \rceil \leftarrow \min \left(\lceil \text{Time}_j \rceil, \max_{v_i \in \vec{s}.\text{getInsert}(v_j), v_i \rightarrow v_k} \lceil \text{Time}_k \rceil - s_j - d_{j,k} \right) \quad (73)$$

The visit time of such nodes is updated based on their earliest predecessor and latest successor in (72) and (73), respectively. The time complexity of the filtering is the same as in the Distance constraint: $O(|E|)$. Although we reason over a set of required nodes as in (Thomas et al. 2020), we do not ensure that a valid transition exists among all required nodes: this problem is NP-complete and would be too computationally expensive to perform at every filtering.

C.2 Precedence

The filtering consists of two main steps.

- (1) First, it considers the partial sequence $\vec{s}$ and ensures that nodes belonging to $\vec{o} \cap \vec{s}$ appear in the same order in both $\vec{o}$ and $\vec{s}$. This step can be implemented in $O(\max(|\vec{o}|, |\vec{s}|))$ and may trigger a failure.
- (2) Next, it considers the insertable nodes from the ordering $\vec{o}$ to respect, removing insertions that would violate the ordering if performed. This is described in Algorithm 9. A set Q tracks the nodes whose insertions will be filtered. The main loop iterates over each node $v_k \in \vec{o}$, as well as the last node ω . If v_k is insertable (*i.e.*, it is not yet in $\vec{s}$), it is added to Q for potential filtering (line 5). On the contrary, if v_k is already in $\vec{s}$, it serves as a boundary for the nodes in Q . For each node v_j in Q , we enforce that it can only be inserted between v_i and v_k , where:
 - v_i is the previous node in $\vec{o}$ that also belongs to $\vec{s}$, found in a previous iteration.
 - v_k (currently being iterated over) is the next node after v_j in $\vec{o}$ that belongs to $\vec{s}$.
 To enforce this consistency, we forbid all sequences where v_j appears before v_i or after v_k (line 7), preserving only insertions consistent with the required precedence.

Algorithm 9: Precedence($\vec{s}, \vec{o}$) constraint filtering for invalid insertions.

```

1  $Q \leftarrow \emptyset$ 
2  $v_i \leftarrow \alpha$ 
3 for  $v_k \in \vec{o} \cdot \omega$  do
4   if  $\vec{s}.\text{isInsertable}(v_k)$  then
5      $Q \leftarrow Q \cup \{v_k\}$ 
6   else if  $\vec{s}.\text{isMember}(v_k)$  then
7     for  $v_j \in Q$  do
8        $\vec{s}.\text{notBetween}(\alpha, v_j, v_i)$ 
9        $\vec{s}.\text{notBetween}(v_k, v_j, \omega)$ 
10     $Q \leftarrow \emptyset$ 
11     $v_i \leftarrow v_k$ 

```

The time complexity of the filtering is dominated by Algorithm 9, running in $O(|\vec{o}| \cdot |V|)$.

C.3 Cumulative

Firstly, to ensure that the start p_i and the end d_i of an activity $i \in A$ are visited together and in a valid order, two constraints are added per request. The constraint $\mathcal{R}_{p_i}(\vec{s}) = \mathcal{R}_{d_i}(\vec{s})$ ensures that the two nodes p_i, d_i appear together, and Precedence($\vec{s}, (p_i \cdot d_i)$) ensures that the start p_i appears before the end d_i . The remaining filtering consists of three steps: computing a load profile, filtering the partially inserted activities and then the non-inserted activities.

A lower bound on the load profile is computed based on the activities that are fully or partially inserted, representing the sum of resources consumed at each node. It is described by three values for each inserted node $v \in \vec{s}$:

- l_v^- for the accumulated load before the visit of v ;
- l_v^+ for the accumulated load at the visit of v ;
- l_v^{o+1} for the accumulated load between the visit of v and its successor.

A load profile example is shown in Figure 8. Using three values per node to represent the load profile is needed to accurately represent whether activities may be put between a node and its successor, as shown in Figure 9.

For each fully inserted activity i , the load of i is added between the nodes: $\forall p_i \prec v \preceq d_i : I_v^- \leftarrow I_v^- + I_i$, $\forall p_i \preceq v \prec d_i : I_v^+ \leftarrow I_v^+ + I_i \wedge I_v^{v+1} \leftarrow I_v^{v+1} + I_i$. When considering only the inserted activities, it follows that $\forall v \in \vec{s} : I_v^+ = I_v^{v+1}$. Those equalities no longer hold after considering the partially inserted activities.

For partially inserted activities with a start inserted, a node from the partial sequence $\vec{s}$ is used instead of the non-inserted end node to compute the load. It corresponds to the earliest node after the start node or the start node itself, after which the non-inserted end can be inserted. Partially inserted activities with the end inserted behave similarly, considering the latest node preceding the end, after which the start node can be inserted.

Setting an entry in I^- , I^+ or I^{+1} exceeding the capacity triggers a failure.

Given the load profile, we filter insertions for the partially inserted activities and the non-inserted activities. Algorithm 10 depicts the filtering for every non-inserted activity $i \in A$ whose start p_i is inserted but not its corresponding end d_i . It first finds the closest node v after which the end d_i can be inserted (line 3), triggering a failure if no such node exists (line 6). Note that this closest node v was already used to set the load of activity i during the load profile computation, and therefore is already marked as valid (line 7). The next nodes to inspect are therefore the nodes following v , in order. As soon as the capacity occurring at a node v does not allow inserting the end d_i of the activity, all insertions between this invalid node v and the end ω of the sequence are removed (line 9). A similar filtering is performed in a mirror fashion, considering the non-inserted activities whose ends are inserted but not the corresponding start.

Finally, the filtering from (Thomas et al. 2020) is used for non-inserted activities, inspecting every start (and end) of activities, checking if a matching end (and start) can be found, and removing insertions when no match exists.

Algorithm 10: Filtering the Cumulative($\vec{s}, p, d, l, c$) constraint for activities whose starts are inserted.

Input: $\vec{s}$: sequence variable, p, d, l : start, end and load of activities, c : capacity.

```

1 for  $i \in A$  s.t.  $(\vec{s}.\text{isMember}(p_i) \text{ and not } \vec{s}.\text{isMember}(d_i))$  do
2    $v \leftarrow p_i$ 
3   while not  $\vec{s}.\text{canInsert}(v, d_i)$  do
4      $v \leftarrow \vec{s}.\text{getNext}(v)$ 
5     if  $v = \omega$  then
6       return failure
7    $v \leftarrow \vec{s}.\text{getNext}(v)$ 
8   while  $v \neq \omega$  do
9     if  $\max(I_v^-, I_v^+) + l_i > c$  then
10        $\vec{s}.\text{notBetween}(v, d_i, \omega)$ 
11       break
12    $v \leftarrow \vec{s}.\text{getNext}(v)$ 

```

C.4 SubSequence

The SubSequence constraint links two sequence variables $\vec{s}_m, \vec{s}_s$, ensuring that a subsequence $\vec{s}_s$ is contained within a supersequence (or master sequence) $\vec{s}_m$.

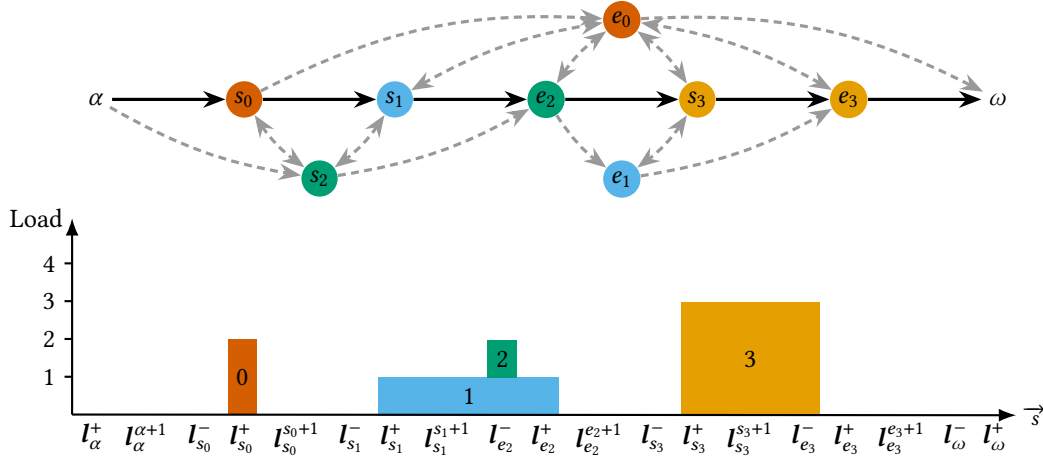


Fig. 8. A sequence variable $\vec{S}$ with partial sequence $\vec{s} = \alpha \cdot s_0 \cdot s_1 \cdot e_2 \cdot s_3 \cdot e_3 \cdot \omega$ (top) and its corresponding load profile (bottom) when using a Cumulative($\vec{S}, (s_0, s_1, s_2, s_3), (e_0, e_1, e_2, e_3), (2, 1, 1, 3), 4$) constraint. Edges between insertable nodes are not shown. For the partially inserted activity 1, the closest node after which e_1 can be inserted is e_2 ; hence, rectangle 1 ends at $I_{e_2}^+$ instead of $I_{s_1}^+$ on the load profile.

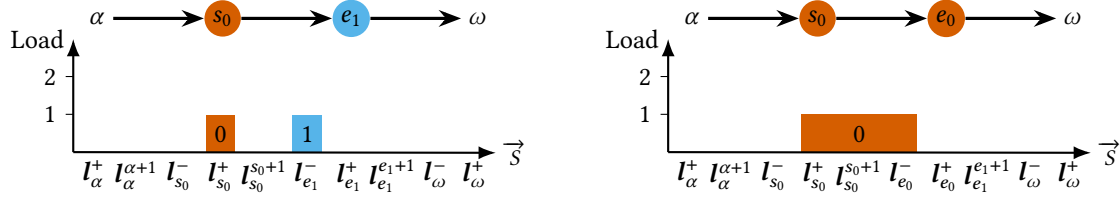


Fig. 9. Two load profiles for different sequences, with a capacity of 2. With $I_{s_0}^{s_0+1}$, we detect that an activity of load 2 can be put between s_0 and e_1 on the left but cannot be put between s_0 and e_0 on the right.

$$\text{SubSequence}(\vec{S}_m, \vec{S}_s) \leftrightarrow \left(v_i \in \vec{S}_s \implies v_i \in \vec{S}_m \right) \wedge \left(\forall v_i, v_j \in \vec{S}_s : v_i \prec_{\vec{S}_s} v_j \implies v_i \prec_{\vec{S}_m} v_j \right) \quad (74)$$

Filtering. Firstly, to ensure that every node v_i within the subsequence $\vec{S}_s$ is visited by the master sequence $\vec{S}_m$, implication constraints are used:

$$\mathcal{R}_i(\vec{S}_s) \implies \mathcal{R}_i(\vec{S}_m) \quad \forall v_i \in V \quad (75)$$

where V is the set of nodes over which the subsequence $\vec{S}_s$ is defined.

Apart from those implications, the filtering is essentially the same as the one performed by the Precedence constraint from Section 7.3. The partial sequences $\vec{s}_m, \vec{s}_s$ of the two sequence variables $\vec{S}_m, \vec{S}_s$ are retrieved, and the filtering ensures that nodes in common to the two partial sequences appear in the same order:

$$\forall v_i, v_j \in \vec{S}_s \cap \vec{S}_m : v_i \prec_{\vec{S}_s} v_j \iff v_i \prec_{\vec{S}_m} v_j \quad (76)$$

If (76) does not hold, a failure is triggered. Then, filtering of detour edges is performed by calling twice Algorithm 9 from the Precedence constraint. It is called once with $\vec{S} = \vec{S}_m$ and $\vec{d} = \vec{d}_s$, enforcing precedences from the partial subsequence $\vec{s}_s$ onto the master sequence $\vec{S}_m$, and a second time with $\vec{S} = \vec{S}_s$ and $\vec{d} = \vec{d}_m$, enforcing precedences from the partial master sequence $\vec{s}_m$ onto the subsequence $\vec{S}_s$.

D DARP

In both the sequence model and the successor model, the set of nodes V and the distance matrix d are extended to introduce nodes representing the start and end nodes of the vehicles, which are duplicates of the depot. The values of C_1, C_2 in (39) were taken from (Jain and Van Hentenryck 2011) and set to 80 and 1, respectively.

Search annotations were provided for the successor model, as they greatly improve the quality of solutions. They specify that branching first considers the successor variables **Succ** using the DomWDeg heuristic (Boussemart, Hemery, et al. 2004), which prioritizes variables associated with small domains and a large number of encountered failures. The LNS consists of fixing 85% of variables to their values in a previous solution and is restarted after a constant number of 10,000 search nodes.

D.1 Results per instance

Table 6 shows the performance of the different solvers for each instance. For the RF (2021) method from (Rist and Forbes 2021), we directly used the authors' implementation. The solver managed to provide a proof of optimality within 57s, 14m25s and 14m30s on average on R1a, R1b and R7a, respectively, but failed to provide any feasible solution on the other instances as the enumeration of the fragments of paths timed out.

Table 6. Primal gaps (%) to the best known solution (bks) on DARP instances after 15 minutes of runtime. Entries report the mean primal gap $\pm$ the sample standard deviation and the best primal gap over ten runs. A dash indicates that no run found a feasible solution. Except for entries marked with † , numerical entries are based on ten feasible runs. For marked entries, statistics use the feasible runs specified in the table note. Best mean and best gap values are in bold.

Name	k	$ R $	bks	Succ		Succ-LNS		CPO		RF (2021)		OR-Tools Routing		Hexaly		Seqvar	
				Mean $\pm$ SD	Best	Mean $\pm$ SD	Best	Mean $\pm$ SD	Best	Mean $\pm$ SD	Best	Mean $\pm$ SD	Best	Mean $\pm$ SD	Best	Mean $\pm$ SD	Best
R1a	3	24	190.02	18.26 $\pm$ 0.00	18.26	6.03 $\pm$ 2.00	3.19	3.62 $\pm$ 1.54	0.71	0.00 $\pm$ 0.00	0.00	-	-	2.98 $\pm$ 1.42	0.54	0.00 $\pm$ 0.00	0.00
R1b	3	24	164.46	33.22 $\pm$ 0.00	33.22	7.81 $\pm$ 3.66	4.34	12.81 $\pm$ 4.15	4.32	0.00 $\pm$ 0.00	0.00	0.00 $\pm$ 0.00	0.00	3.69 $\pm$ 1.33	1.38	0.82 $\pm$ 1.35	0.00
R7a	4	36	291.71	-	-	11.22 $\pm$ 4.71	4.71	23.55 $\pm$ 14.85	6.09	0.00 $\pm$ 0.00	0.00	-	-	4.39 $\pm$ 2.03	1.08	2.06 $\pm$ 1.34	0.00
R7b	4	36	248.21	-	-	10.87 $\pm$ 1.96	7.33	12.95 $\pm$ 3.48	8.11	-	-	-	-	8.58 $\pm$ 2.32	5.75	2.41 $\pm$ 1.46	1.07
R2a	5	48	301.34	-	-	8.03 $\pm$ 2.79	2.87	29.61 $\pm$ 24.07 †	12.59	-	-	-	-	6.70 $\pm$ 4.01	2.39	1.55 $\pm$ 1.05	0.29
R2b	5	48	295.66	44.39 $\pm$ 0.00	44.39	12.40 $\pm$ 3.24	7.33	15.76 $\pm$ 2.11	11.88	-	-	1.46 $\pm$ 0.00	1.46	8.07 $\pm$ 2.22	4.77	2.41 $\pm$ 0.80	0.76
R8a	6	72	487.84	-	-	-	-	-	-	-	-	-	-	14.91 $\pm$ 3.43	10.81	6.45 $\pm$ 1.63	3.73
R8b	6	72	458.73	-	-	24.17 $\pm$ 2.77	20.69	-	-	-	-	10.15 $\pm$ 0.00	10.15	12.48 $\pm$ 2.08	8.62	7.06 $\pm$ 2.14	3.39
R3a	7	72	532.00	-	-	-	-	-	-	-	-	-	-	10.21 $\pm$ 2.73	5.23	4.04 $\pm$ 1.13	2.30
R3b	7	72	484.83	-	-	-	-	50.86 $\pm$ - †	50.86	-	-	5.75 $\pm$ 0.00	5.75	12.15 $\pm$ 2.01	9.38	6.03 $\pm$ 0.97	4.97
R9a	8	108	653.94	-	-	-	-	-	-	-	-	-	-	27.42 $\pm$ 2.52	22.50	7.18 $\pm$ 1.84	3.99
R9b	8	108	592.23	-	-	-	-	-	-	-	-	-	-	16.05 $\pm$ 3.11	13.02	8.72 $\pm$ 1.75	6.10
R4a	9	96	570.25	-	-	-	-	-	-	-	-	9.62 $\pm$ 0.00	9.62	16.61 $\pm$ 2.53	12.26	7.82 $\pm$ 1.56	3.80
R4b	9	96	529.33	-	-	-	-	36.66 $\pm$ 13.98	25.12	-	-	11.33 $\pm$ 0.00	11.33	16.44 $\pm$ 2.17	12.43	9.12 $\pm$ 2.05	5.78
R10a	10	144	845.47	-	-	-	-	-	-	-	-	-	-	25.56 $\pm$ 3.73	20.74	10.77 $\pm$ 1.09	9.12
R10b	10	144	783.81	-	-	-	-	-	-	-	-	-	-	25.55 $\pm$ 3.81	17.25	11.68 $\pm$ 1.46	8.89
R5a	11	120	625.64	-	-	-	-	-	-	-	-	10.06 $\pm$ 0.00	10.06	15.52 $\pm$ 2.15	12.42	9.64 $\pm$ 1.36	7.75
R5b	11	120	573.56	-	-	-	-	-	-	-	-	9.38 $\pm$ 1.14	7.73	14.66 $\pm$ 2.23	12.34	8.21 $\pm$ 1.52	5.91
R6a	13	144	783.78	-	-	-	-	-	-	-	-	10.99 $\pm$ 0.34	10.03	18.07 $\pm$ 2.88	14.00	10.60 $\pm$ 1.82	8.58
R6b	13	144	725.22	-	-	-	-	-	-	-	-	13.36 $\pm$ 0.59	11.67	16.21 $\pm$ 2.43	12.87	9.76 $\pm$ 1.65	7.40

Note. † CPO on R2a: 2 feasible runs; CPO on R3b: 1 feasible run. For marked entries, the mean, sample standard deviation, and best gap are computed only over the stated feasible runs. The sample standard deviation for CPO on R3b is undefined for one feasible run and is shown as -.

E Prize Collecting Scheduling Problem

Instance selection. The benchmark set of (Froger and Sadykov 2023) contains two datasets, denoted A_E (Avionic) and T (Therapy). For every combination of dataset, number of tasks, and number of secondary resources, 30 instances are available, identified by a numerical suffix ranging from __000 to __029. The Avionic instances use $m \in \{3, 4\}$ secondary resources, whereas the Therapy instances use $m \in \{2, 3\}$.

We retained the task counts $n \in \{50, 100, 150, 200, 250, 300, 400, 500\}$. Instances with 350 and 450 tasks were excluded because the reference results used to compute the primal gaps do not provide best-known values for these sizes. Within each (dataset, n , m) subgroup, instances were ordered by their numerical suffix and three approximately evenly spaced instances were selected: __000, __014, and __029. The selection therefore depends only on the instance identifiers and was performed independently of the results obtained by the compared methods. This gives $2 \times 8 \times 2 \times 3 = 96$ instances.

Table 7 shows the performance of the different solvers for each instance size. Each row corresponds to the average across 3 instances. We compare to the best known solution from (Froger and Sadykov 2023). One should note that the CPO model managed to improve the best known solutions on 10 instances from the T dataset.

Table 7. Primal gaps (%) to the best known solution (bks) on PCSP instances. Entries report the mean primal gap $\pm$ the sample standard deviation across constituent instance-level mean gaps; best gaps retain the existing mean of per-instance best gaps. A dash indicates that no feasible solution was found for the corresponding group. Except for entries marked with † , numerical entries use all ten runs of each constituent instance. For marked entries, the affected instance contribution uses its feasible runs only, as specified in the table note. Best mean and best gap values are in bold.

Dataset	n	m	MZN-Succ		MZN-Disj		CPO		OR-Tools Routing		Hexaly		SeqVar	
			Mean $\pm$ SD	Best	Mean $\pm$ SD	Best	Mean $\pm$ SD	Best	Mean $\pm$ SD	Best	Mean $\pm$ SD	Best	Mean $\pm$ SD	Best
A_E	50	3	17.11 $\pm$ 2.19	16.40	19.34 $\pm$ 0.84	19.25	0.00 $\pm$ 0.00	0.00	4.25 $\pm$ 1.59	4.25	32.30 $\pm$ 3.54	25.26	0.00 $\pm$ 0.00	0.00
		4	18.35 $\pm$ 1.94	17.81	20.02 $\pm$ 3.03	19.86	0.11 $\pm$ 0.18	0.00	3.81 $\pm$ 1.35	3.81	31.41 $\pm$ 2.59	28.11	0.00 $\pm$ 0.00	0.00
	100	3	61.79 $\pm$ 8.10	61.75	36.24 $\pm$ 3.76	36.24	0.75 $\pm$ 0.90	0.00	4.62 $\pm$ 0.29	4.62	39.28 $\pm$ 1.40	35.04	0.41 $\pm$ 0.29	0.28
		4	77.99 $\pm$ 11.88	77.99	56.11 $\pm$ 38.28	56.11	0.70 $\pm$ 0.74	0.00	3.93 $\pm$ 1.09	3.93	40.80 $\pm$ 3.84	37.13	0.32 $\pm$ 0.43	0.00
	150	3	96.44 $\pm$ 6.17	96.44	44.42 $\pm$ 2.57	44.29	0.36 $\pm$ 0.53	0.28	3.83 $\pm$ 2.80	3.83	45.59 $\pm$ 1.73	38.84	0.59 $\pm$ 0.42	0.14
		4	86.85 $\pm$ 4.92	86.85	45.05 $\pm$ 0.92	45.05	0.32 $\pm$ 0.32	0.00	5.09 $\pm$ 6.36	5.09	44.06 $\pm$ 1.35	40.54	0.20 $\pm$ 0.30	0.11
	200	3	79.93 $\pm$ 24.92	79.93	49.19 $\pm$ 0.79	49.19	0.45 $\pm$ 0.29	0.00	2.94 $\pm$ 1.48	2.94	47.28 $\pm$ 1.17	45.14	0.10 $\pm$ 0.08	0.08
		4	93.81 $\pm$ 5.49	93.81	49.08 $\pm$ 1.67	49.08	0.13 $\pm$ 0.12	0.00	5.15 $\pm$ 1.46	5.15	44.64 $\pm$ 0.10	40.32	0.41 $\pm$ 0.52	0.05
	250	3	96.68 $\pm$ 5.75	96.68	50.28 $\pm$ 2.75	50.11	0.22 $\pm$ 0.25	0.00	1.27 $\pm$ 1.81	1.27	47.85 $\pm$ 1.10	42.92	0.18 $\pm$ 0.16	0.00
		4	86.94 $\pm$ 17.62	86.94	51.84 $\pm$ 2.62	51.84	0.66 $\pm$ 0.34	0.21	4.03 $\pm$ 2.70	4.03	45.53 $\pm$ 0.69	42.14	0.22 $\pm$ 0.39	0.21
	300	3	89.93 $\pm$ 11.34	89.93	53.69 $\pm$ 3.29	53.62	0.69 $\pm$ 0.13	0.08	3.48 $\pm$ 0.94	3.48	47.00 $\pm$ 0.96	44.05	0.82 $\pm$ 0.98	0.29
		4	98.70 $\pm$ 2.25	98.70	53.74 $\pm$ 1.81	53.74	0.86 $\pm$ 0.63	0.08	2.87 $\pm$ 1.55	2.87	47.51 $\pm$ 1.62	42.23	1.12 $\pm$ 1.02	0.72
	400	3	99.46 $\pm$ 0.93	99.46	54.31 $\pm$ 4.22	54.08	1.09 $\pm$ 1.07	0.31	2.67 $\pm$ 1.42	2.67	49.53 $\pm$ 0.60	45.85	1.31 $\pm$ 1.09	0.39
		4	97.20 $\pm$ 4.84	97.20	69.70 $\pm$ 26.28	69.70	0.39 $\pm$ 0.19	0.13	1.59 $\pm$ 0.34	1.59	49.16 $\pm$ 0.91	46.18	0.68 $\pm$ 0.29	0.15
	500	3	97.06 $\pm$ 5.09	97.06	71.06 $\pm$ 25.12	70.73	0.79 $\pm$ 0.23	0.00	2.35 $\pm$ 1.86	2.35	51.10 $\pm$ 0.78	48.40	0.90 $\pm$ 0.60	0.45
		4	95.85 $\pm$ 4.77	95.41	72.07 $\pm$ 24.19	72.07	0.50 $\pm$ 0.43	0.18	3.08 $\pm$ 0.77	3.08	49.78 $\pm$ 1.58	46.94	1.41 $\pm$ 0.47	0.54
T	50	2	7.69 $\pm$ 4.58	6.36	0.00 $\pm$ 0.00	0.00	0.00 $\pm$ 0.00	0.00	5.64 $\pm$ 4.29	5.64	13.20 $\pm$ 4.80	7.91	0.00 $\pm$ 0.00	0.00
		3	2.39 $\pm$ 2.49	2.17	0.00 $\pm$ 0.00	0.00	0.00 $\pm$ 0.00	0.00	7.78 $\pm$ 2.18	7.78	12.09 $\pm$ 1.51	8.27	0.00 $\pm$ 0.00	0.00
	100	2	72.27 $\pm$ 22.53	72.01	20.94 $\pm$ 0.59	20.94	0.87 $\pm$ 0.62	0.38	7.46 $\pm$ 2.05	7.46	23.21 $\pm$ 0.10	18.18	0.65 $\pm$ 0.62	0.00
		3	72.78 $\pm$ 2.59	72.78	15.47 $\pm$ 1.88	15.25	0.50 $\pm$ 0.23	0.00	7.56 $\pm$ 2.10	7.56	19.76 $\pm$ 1.20	15.36	0.51 $\pm$ 0.15	0.00
	150	2	91.94 $\pm$ 13.95	91.94	32.49 $\pm$ 1.26	32.49	2.00 $\pm$ 0.70	0.83	10.27 $\pm$ 2.89	10.27	29.88 $\pm$ 3.14	24.95	2.17 $\pm$ 0.87	1.25
		3	82.11 $\pm$ 15.69	79.41	23.56 $\pm$ 2.41	23.56	0.35 $\pm$ 0.36	0.08	6.81 $\pm$ 0.70	6.81	24.17 $\pm$ 0.23	20.17	2.03 $\pm$ 0.39	1.26
	200	2	96.72 $\pm$ 5.68	96.72	43.53 $\pm$ 2.39	43.53	2.08 $\pm$ 0.43	1.52	9.31 $\pm$ 0.76	9.31	32.12 $\pm$ 0.47	27.19	3.21 $\pm$ 0.02	2.20
		3	88.27 $\pm$ 10.35	88.27	35.79 $\pm$ 2.19	35.36	0.62 $\pm$ 0.28	0.27	6.58 $\pm$ 1.09	6.57	25.36 $\pm$ 1.88	21.70	2.10 $\pm$ 0.21	1.44
	250	2	-	-	47.90 $\pm$ 2.61	47.90	1.57 $\pm$ 0.70	0.82	8.44 $\pm$ 0.25	8.44	32.25 $\pm$ 1.70	27.14	2.83 $\pm$ 0.78	1.93
		3	92.84 $\pm$ 7.48	92.79	44.55 $\pm$ 6.00	44.05	0.19 $\pm$ 0.11	0.00	8.29 $\pm$ 1.22	8.29	25.99 $\pm$ 0.80	22.88	2.15 $\pm$ 0.53	1.35
	300	2	96.26 $\pm$ 6.48	96.26	56.80 $\pm$ 9.47 †	53.46	1.39 $\pm$ 0.04	0.81	9.26 $\pm$ 1.72	9.26	31.99 $\pm$ 1.28	27.32	2.44 $\pm$ 0.58	1.47
		3	96.64 $\pm$ 5.73	96.64	68.25 $\pm$ 29.11 †	66.78	0.27 $\pm$ 0.13	0.00	6.59 $\pm$ 0.22	6.59	25.36 $\pm$ 0.68	21.76	2.08 $\pm$ 0.30	1.51
	400	2	-	-	-	-	1.77 $\pm$ 0.13	1.06	8.40 $\pm$ 1.04	8.40	33.40 $\pm$ 4.53	29.98	3.36 $\pm$ 0.31	2.42
		3	96.38 $\pm$ 3.52	96.16	63.25 $\pm$ 31.83	59.61	0.26 $\pm$ 0.10	0.00	8.76 $\pm$ 0.77	8.67	24.83 $\pm$ 1.01	22.41	2.90 $\pm$ 0.44	2.05
	500	2	-	-	-	-	2.14 $\pm$ 0.23	1.46	10.22 $\pm$ 1.87	10.18	33.73 $\pm$ 0.50	31.78	4.05 $\pm$ 0.03	3.33
		3	-	-	-	-	0.69 $\pm$ 0.24	0.34	9.19 $\pm$ 0.41	9.19	25.59 $\pm$ 0.89	22.33	4.20 $\pm$ 0.72	3.28

Note. † MZN-Disj on T ($n=300, m=2$): 4 feasible runs; MZN-Disj on T ($n=300, m=3$): 4 feasible runs. For marked entries, the contribution of the stated instance to the mean, sample standard deviation, and best gap is computed only over its feasible runs.